CS-663
Digital Image Processing (DIP)

Image Segmentation

Suyash P. Awate

Pseudocode

- en.wikipedia.org/wiki/Pseudocode#Mathematical_style_pseudocode
  - Mathematical notation, typically from matrix and set theory,
  - Control structures of a conventional programming language,
  - Natural language descriptions
- Examples

Return $\sum_{k \in S} x_k$

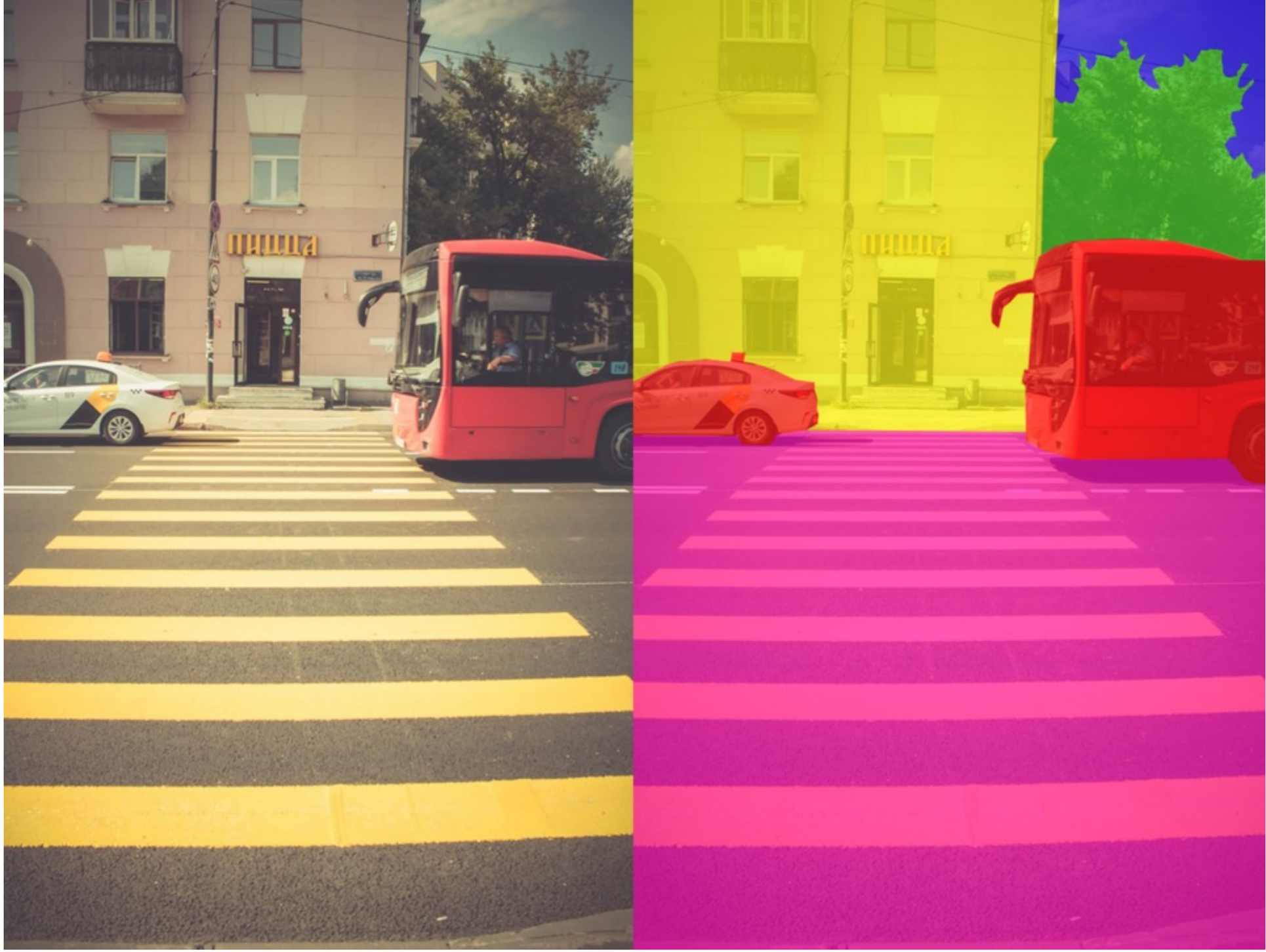
```
algorithm ford-fulkerson is
    input: Graph G with flow capacity c,
           source node s,
           sink node t
    output: Flow f such that f is maximal from s to t

    (Note that f(u,v) is the flow from node u to node v, and c(u,v) is the flow capacity from
node u to node v)

    for each edge (u, v) in GE do
        f(u, v) ← 0
        f(v, u) ← 0

    while there exists a path p from s to t in the residual network Gf do
        let cf be the flow capacity of the residual network Gf
        cf(p) ← min{cf(u, v) | (u, v) in p}
        for each edge (u, v) in p do
            f(u, v) ←  f(u, v) + cf(p)
            f(v, u) ← −f(u, v)

    return f
```

Segmentation

- Partitioning an image into regions or segments
  - Per-pixel labeling task, i.e., task of assigning a label to each pixel
  - Label = categorical variable, e.g., {A, B, C}, {@, #, !}, etc.
  - All pixels with same label = one segment
  - Typically, 1 segment = 1 (or a few) connected regions
- Why ?
  - Simplification of information content
  - Towards higher-level understanding/interpretation
- Criteria
  - Small variation in intensity/texture within segment
  - Large variation in intensity/texture between segments
  - Similar to Otsu thresholding

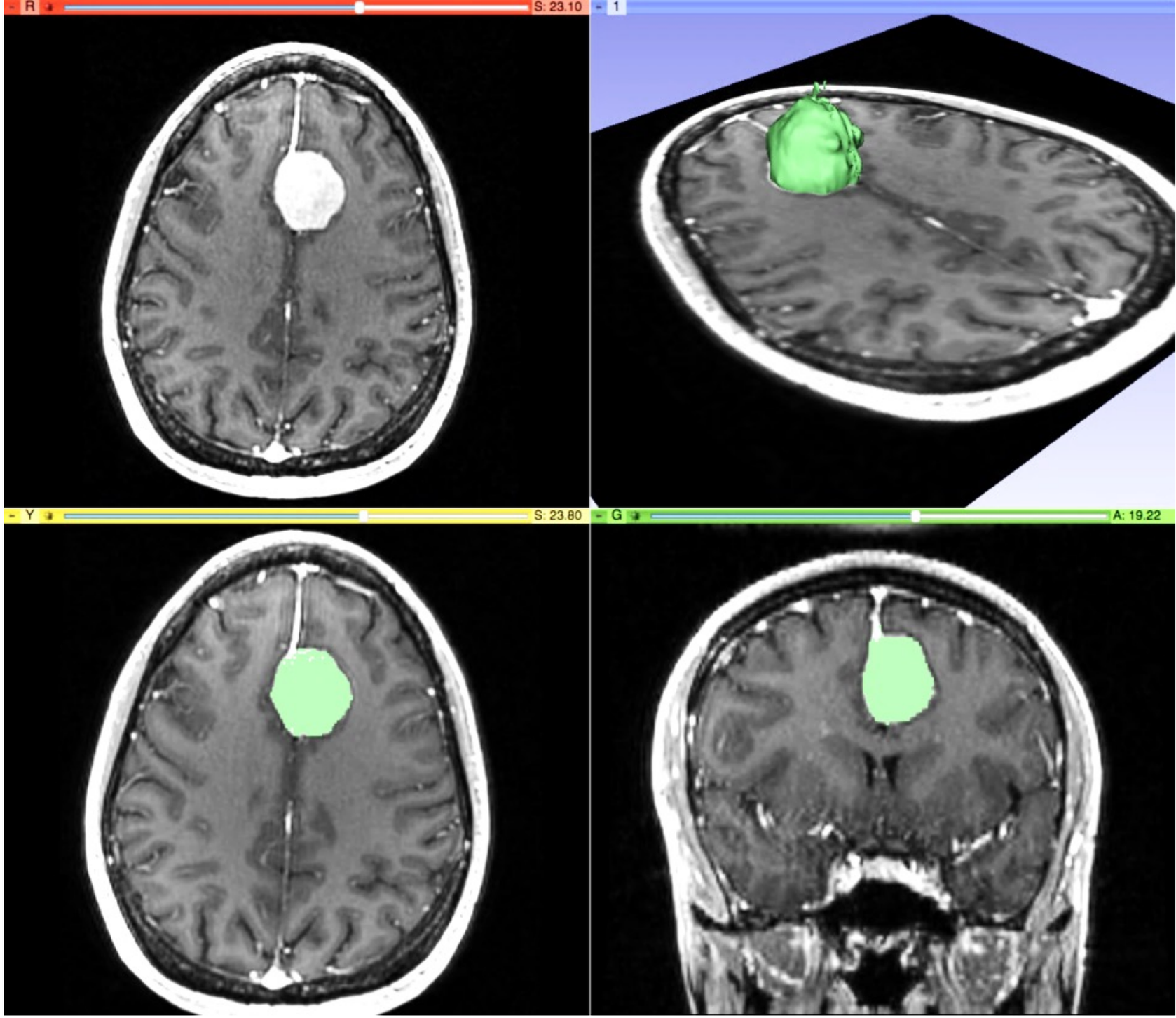
Segmentation

- Example



Segmentation

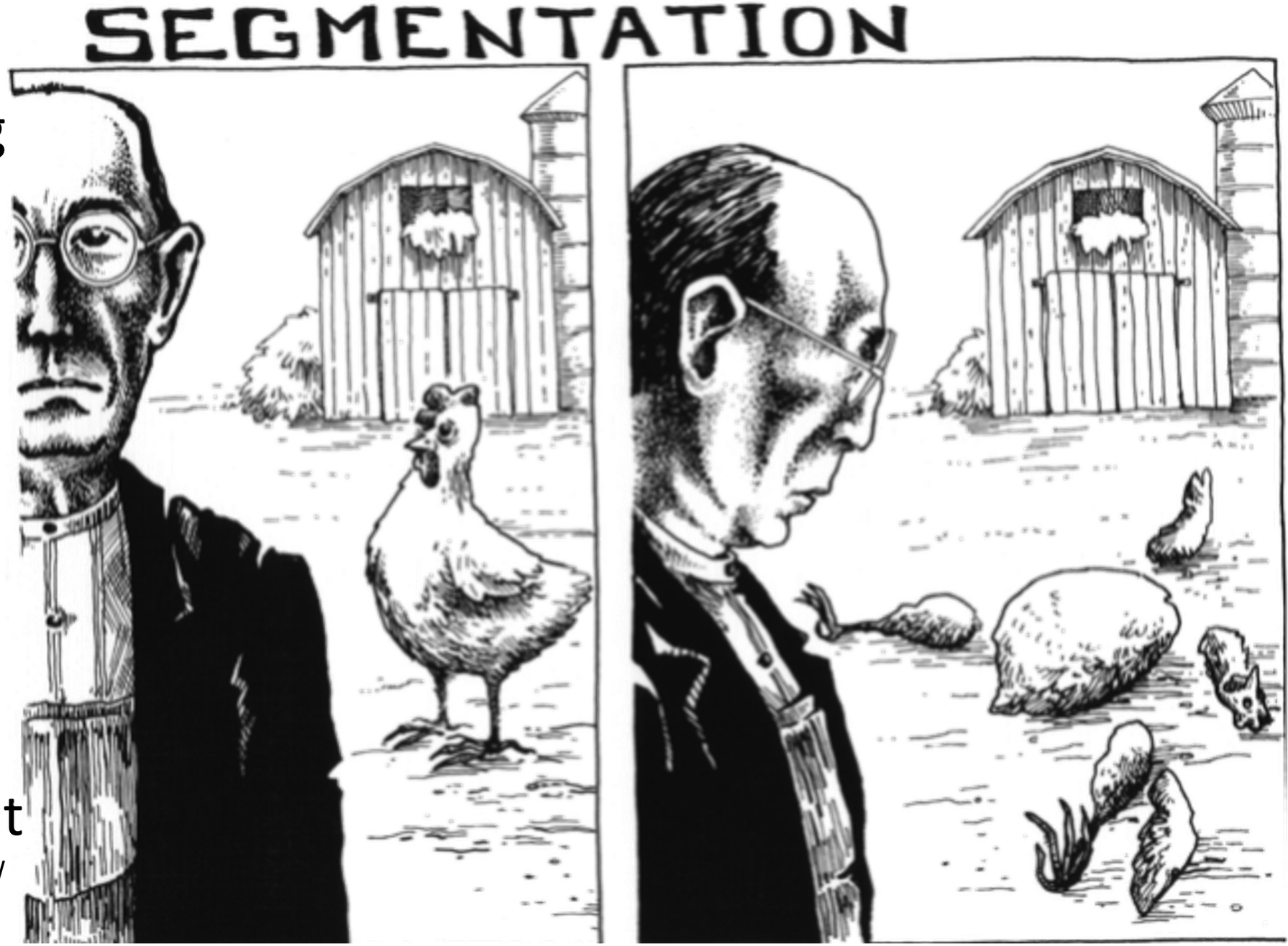
- Top left:
  Original Image
  with tumor
- Bottom: 2D views
  (left: axial,
  right: coronal)
- Top right:
  3D visualization
  of segmented tumor



Segmentation

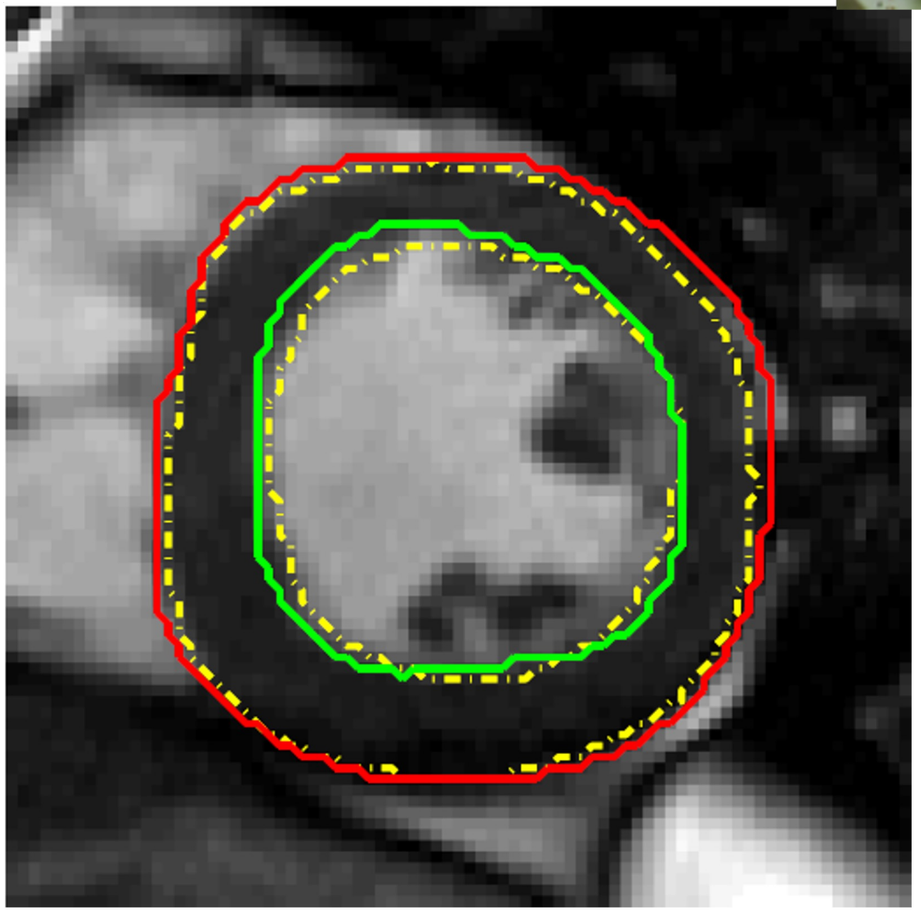
- Human perception
- Instead of perceiving indivisible objects, we perceive objects in terms of their labeled or categorized parts
- We **break** objects **into parts** that we have learned are relevant or important

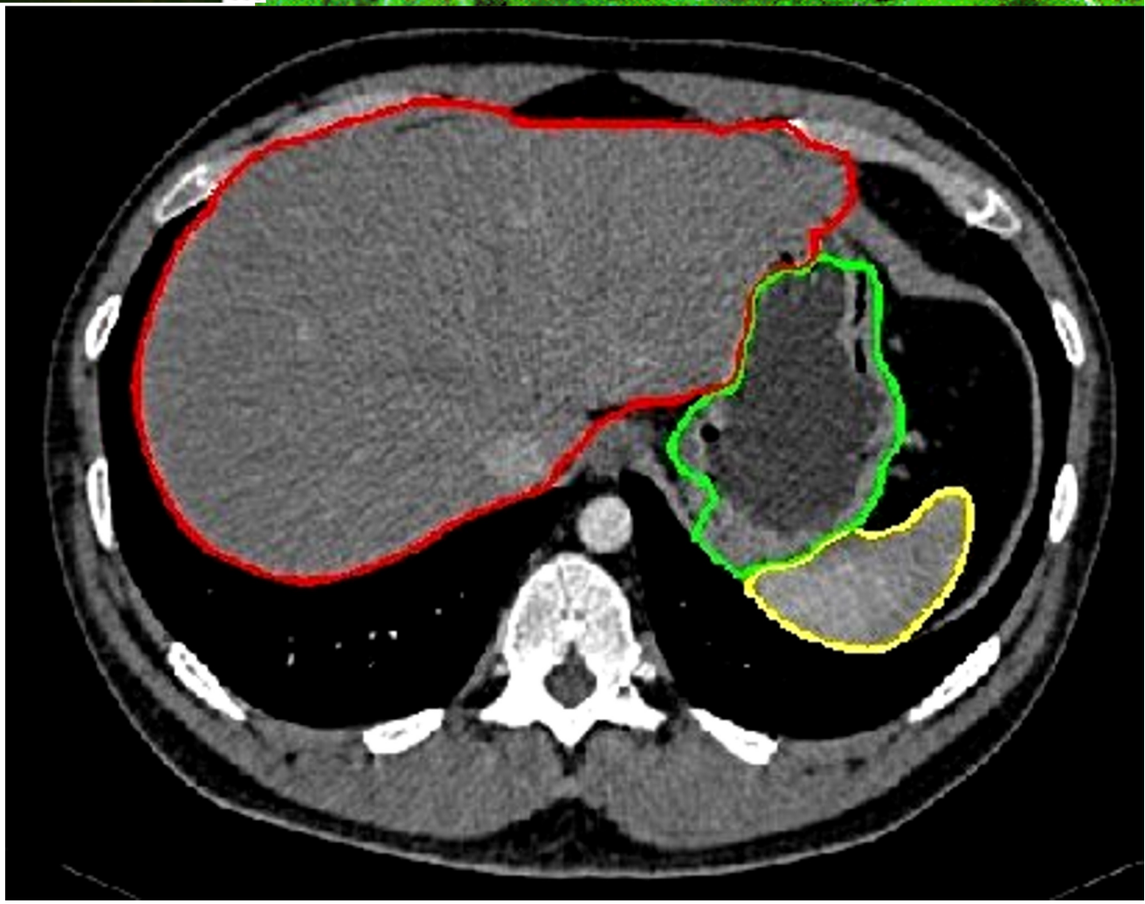
cognitrn.psych.indiana.edu/rgoldsto/projects/segmentation.html



Segmentation

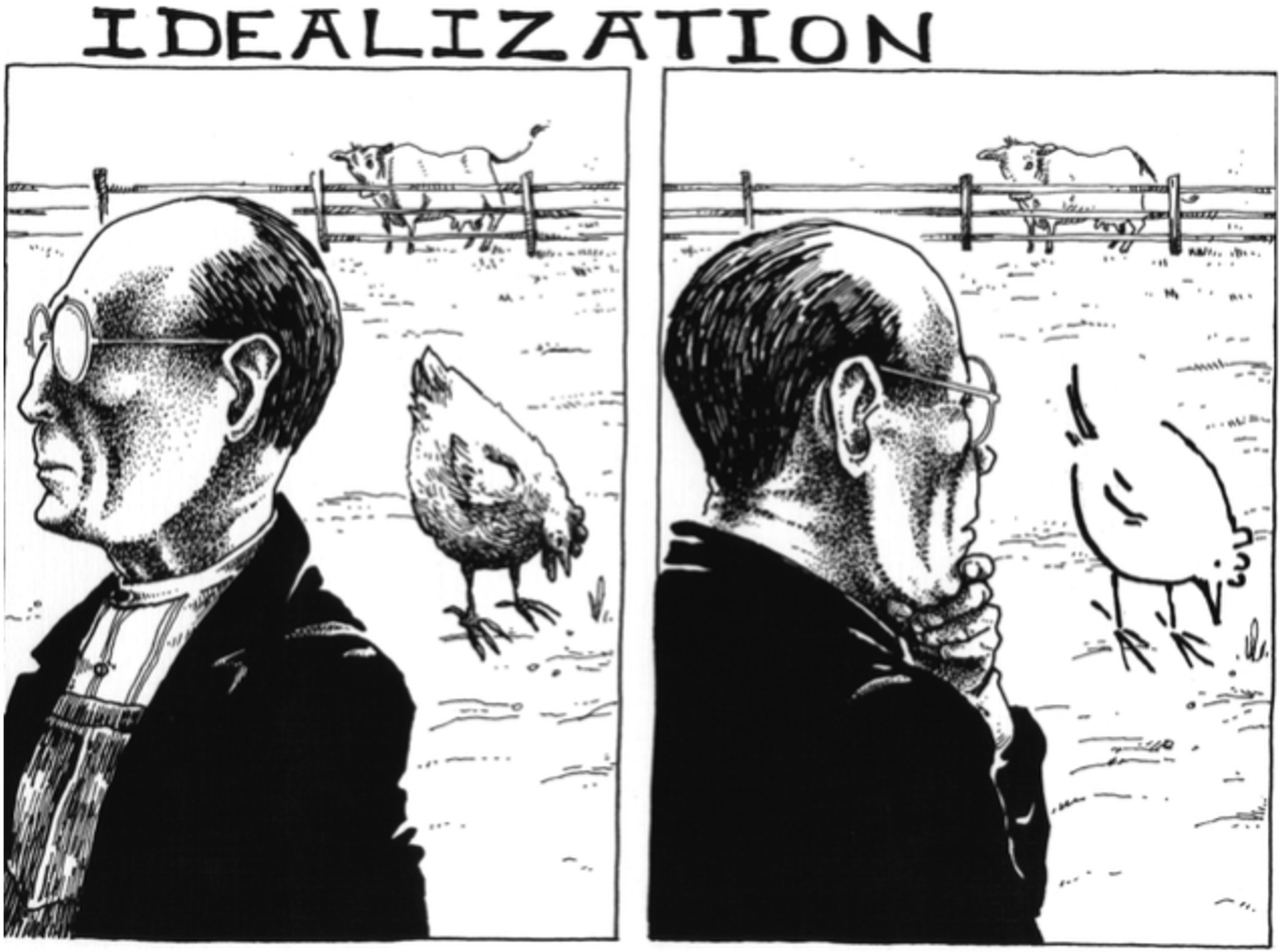
- **Delineation**:
  Process of determining **outline** of an object in image





Segmentation

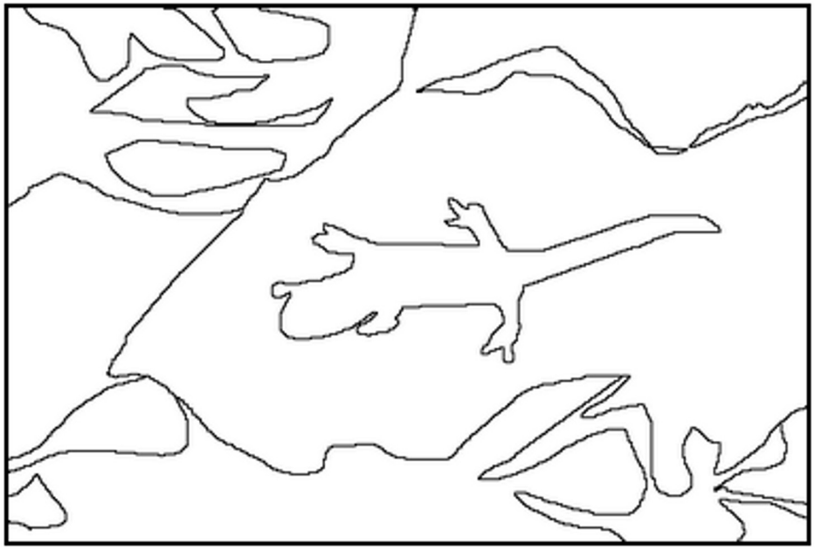
- Human perception (delineation)
- We simplify objects in order to capture basic essence of underlying concept

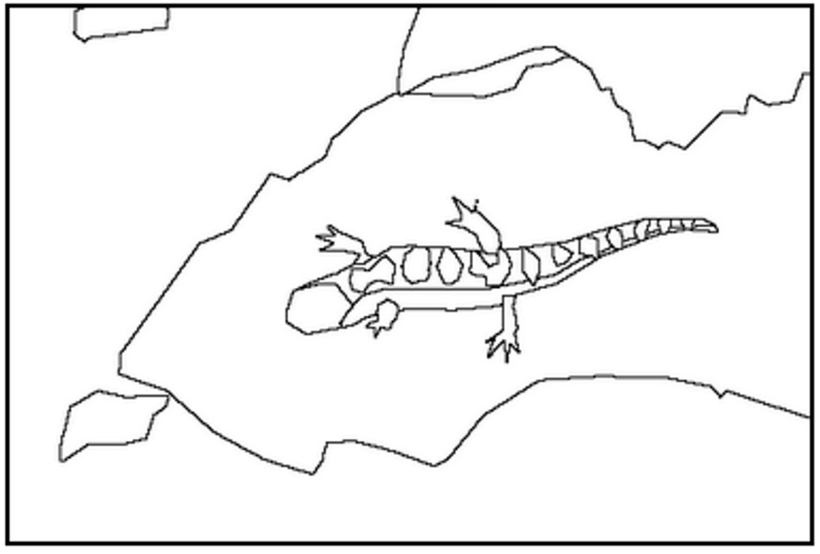


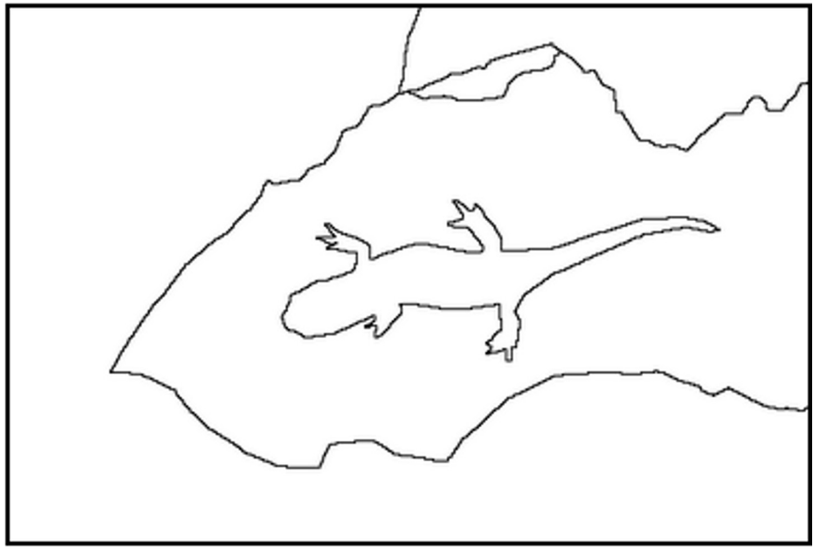
cognitrn.psych.indiana.edu/rgoldsto/projects/idealization.html

Segmentation

- Segmentation and delineation, both, can be subjective
- Different experts can give different answers



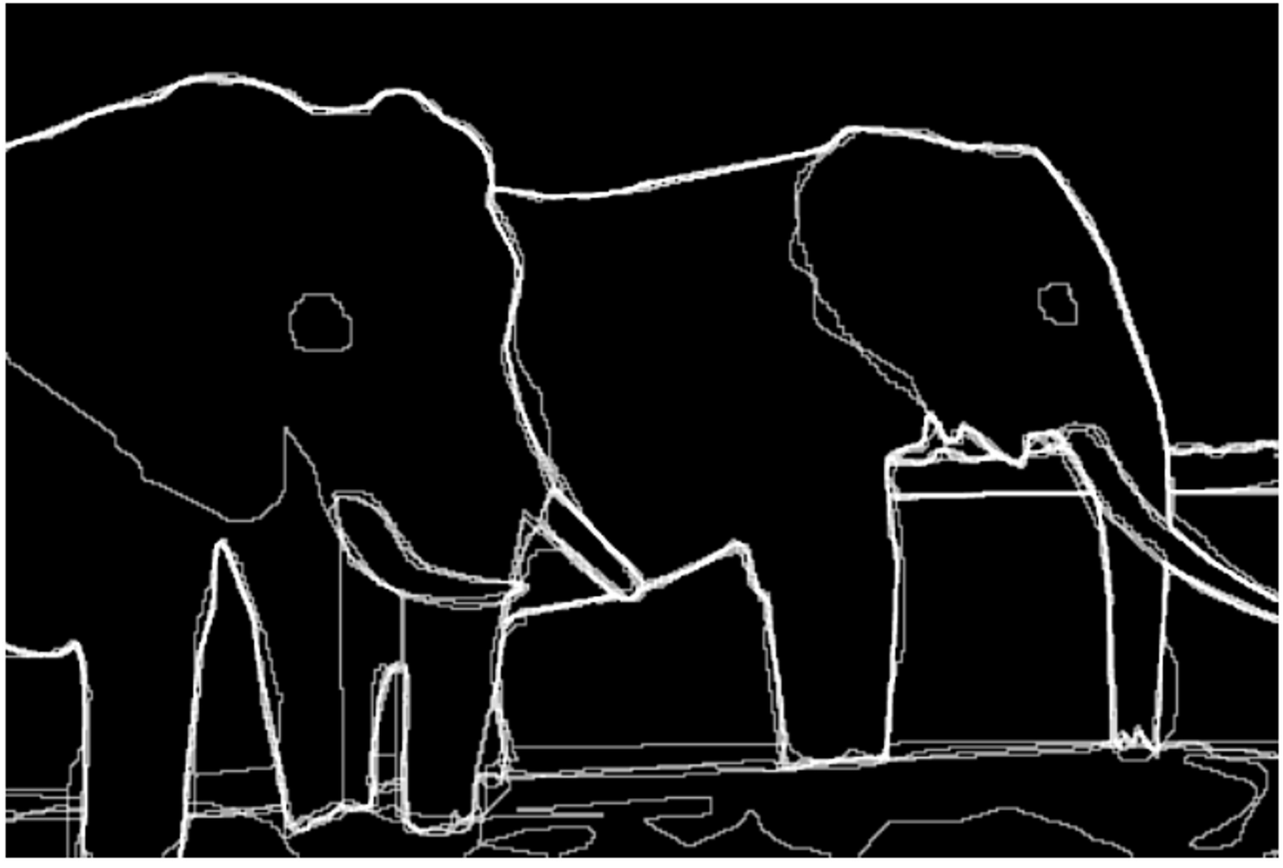




www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/resources.html

Segmentation

- Segmentation and delineation, both, can be subjective
- Different experts can give different answers



Mean-Shift

IEEE TRANSACTIONS ON INFORMATION THEORY, VOL. IT-21, NO. 1, JANUARY 1975

The Estimation of the Gradient of a Density Function, with Applications in Pattern Recognition

KEINOSUKE FUKUNAGA, SENIOR MEMBER, IEEE, AND LARRY D. HOSTETLER, MEMBER, IEEE

***Abstract*—Nonparametric density gradient estimation using a generalized kernel approach is investigated. Conditions on the kernel functions are derived to guarantee asymptotic unbiasedness, consistency, and uniform consistency of the estimates. The results are generalized to obtain a simple mean-shift estimate that can be extended in a *k*-nearest-neighbor approach. Applications of gradient estimation to pattern recognition are presented using clustering and intrinsic dimensionality problems, with the ultimate goal of providing further understanding of these problems in terms of density gradients.**

Mean-Shift Segmentation

IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, VOL. 24, NO. 5, MAY 2002

Mean Shift: A Robust Approach Toward Feature Space Analysis

Dorin Comaniciu, *Member*, *IEEE*, and Peter Meer, *Senior Member*, *IEEE*

Abstract—A general nonparametric technique is proposed for the analysis of a complex multimodal feature space and to delineate arbitrarily shaped clusters in it. The basic computational module of the technique is an old pattern recognition procedure, the mean shift. We prove for discrete data the convergence of a recursive mean shift procedure to the nearest stationary point of the underlying density function and, thus, its utility in detecting the modes of the density. The relation of the mean shift procedure to the Nadaraya-Watson estimator from kernel regression and the robust M-estimators of location is also established. Algorithms for two low-level vision tasks, discontinuity preserving smoothing and image segmentation, are described as applications. In these algorithms, the only user set parameter is the resolution of the analysis and either gray level or color images are accepted as input. Extensive experimental results illustrate their excellent performance.

Index Terms—Mean shift, clustering, image segmentation, image smoothing, feature space, low-level vision.

Density Estimation

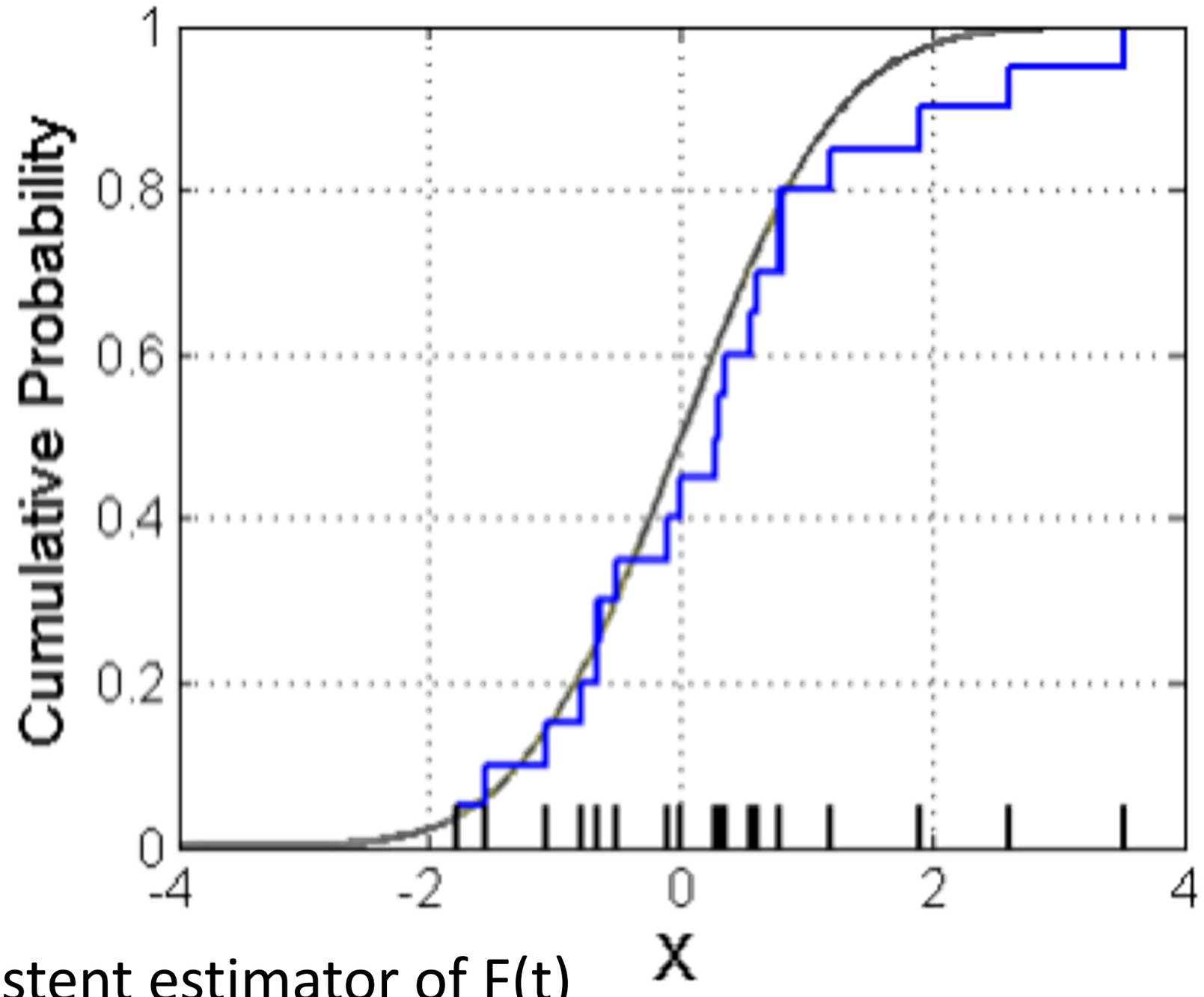
- Goal: Estimate probability density function (PDF) P(X)
  given observations { x_i in R^d : i = 1,…,n } drawn from P(X),
  where analytical/parametric form of P(x) is unknown
- For PDF P(X), let cumulative distribution function (CDF) be F(X)
  - Notation: Let F(t) denote F(X=t)
- Let random variables { X_i ~ P(X): i = 1,…,n } be independent
- **Empirical distribution function (EDF).**
  EDF(t) estimates CDF(t) by counting fraction of n draws ≤ t

$$\widehat{F}_n(t) = \frac{\text{number of elements in the sample} \leq t}{n} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}_{X_i \leq t}$$

- Indicators { $1_{X_i \leq t}$: for all i } are i.i.d. Bernoulli with parameter F(t)

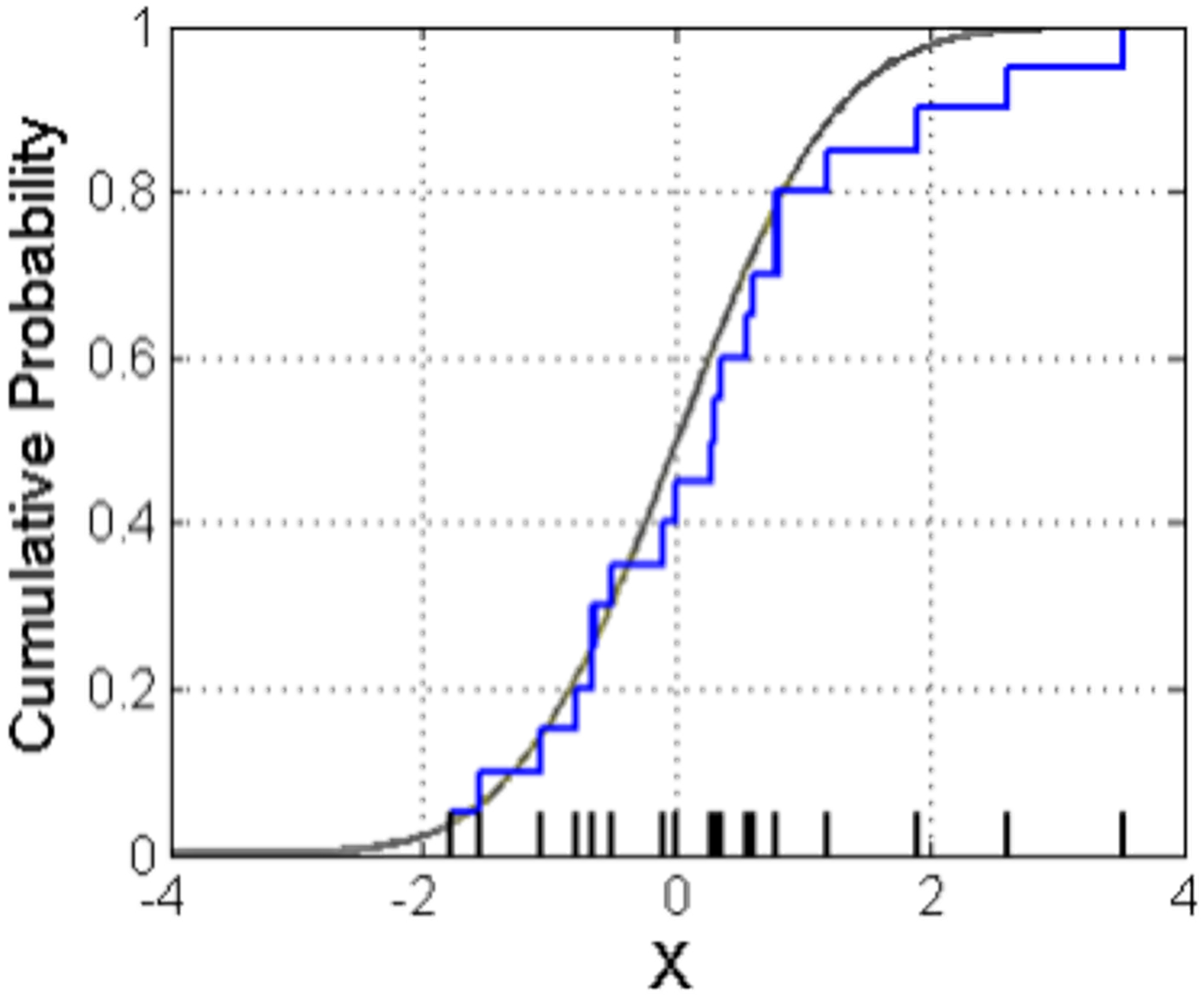
Density Estimation

- $n\widehat{F}_n(t)$ = sum of i.i.d. Bernoulli variables
- $n\widehat{F}_n(t)$ is Binomial; mean nF(t); variance nF(t)(1–F(t))
- As n→∞, $\widehat{F}_n(t)$ has mean = nF(t) / n = F(t); variance = nF(t)(1–F(t)) / n^2 → 0
- So $\widehat{F}_n(t)$ is unbiased consistent estimator of F(t)



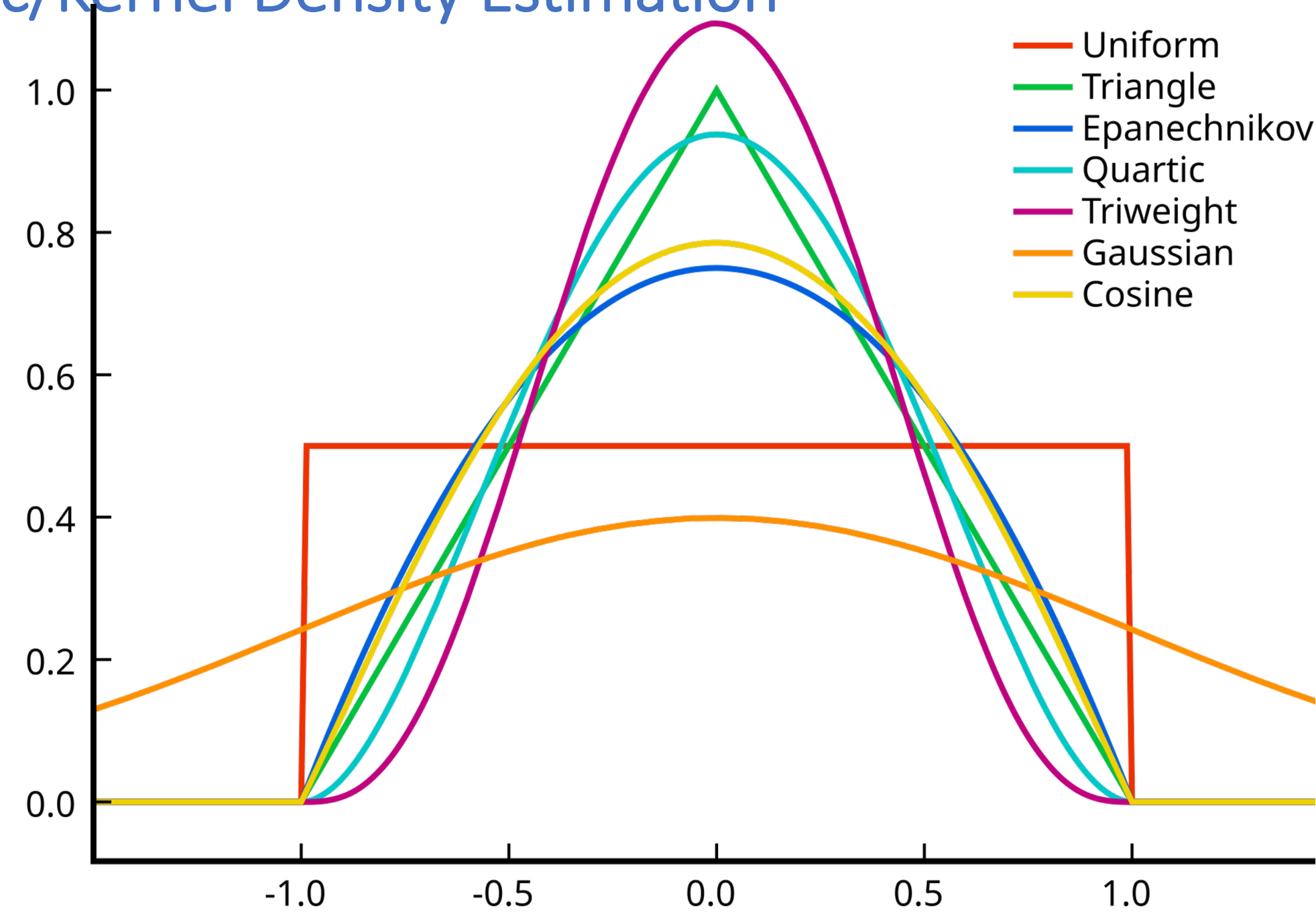
Density Estimation

- EDF is piecewise-constant
- Consider derivative of EDF to approximate PDF
- Derivative = sum of shifted impulses $\delta_{x_i}(x)$; non-smooth
- If we convolve derivative with **smooth** kernel function k(x):
- $\left(\delta_{x_i} * k\right)(y)$
- $:= \int \delta(x - x_i)k(y - x)dx$
- $= \int \delta(z)k(y - x_i - z)dz = k(y - x_i)$ → kernel k(.) shifted by x_i
- Then we get: mixture of kernels → kernel density estimation



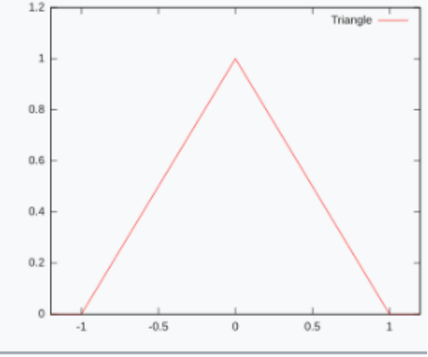
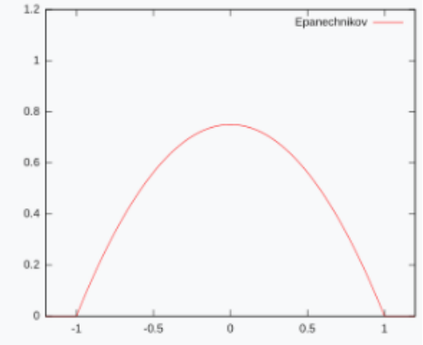
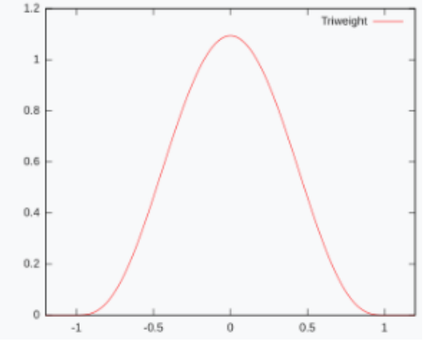
Nonparametric/Kernel Density Estimation

- Common kernel functions



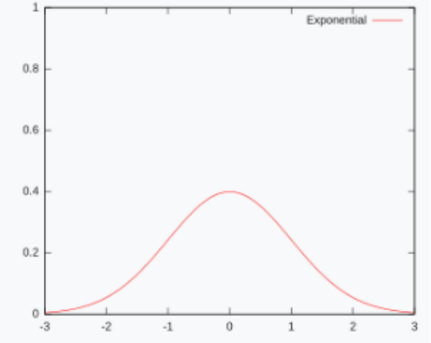
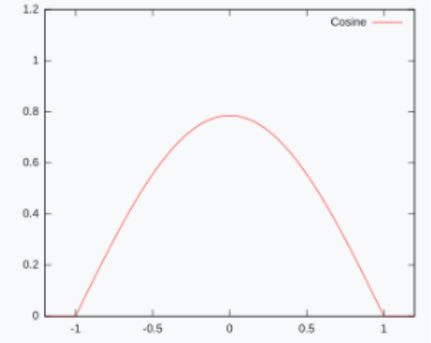
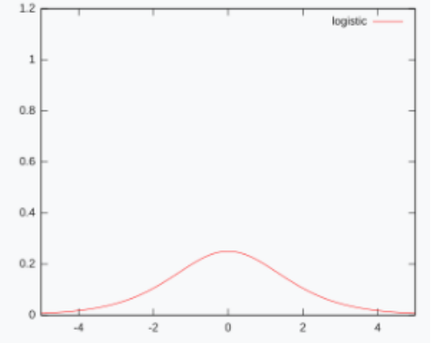
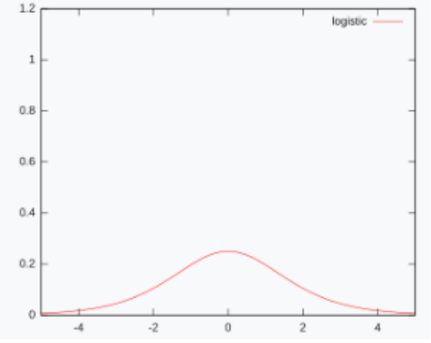
Mean-Shift Segr

- Finite-support kernels

Triangular	$K(u) = (1 - \lvert u \rvert)$ Support: $\lvert u \rvert \leq 1$	
Epanechnikov **(parabolic)**	$K(u) = \frac{3}{4}(1 - u^2)$ Support: $\lvert u \rvert \leq 1$	
Quartic **(biweight)**	$K(u) = \frac{15}{16}(1 - u^2)^2$ Support: $\lvert u \rvert \leq 1$	
Triweight	$K(u) = \frac{35}{32}(1 - u^2)^3$ Support: $\lvert u \rvert \leq 1$	

Mean-Shift Segm

- Kernel functions

Gaussian	$K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2}$	
Cosine	$K(u) = \frac{\pi}{4} \cos\left(\frac{\pi}{2}u\right)$ Support: $\lvert u \rvert \leq 1$	
Logistic	$K(u) = \frac{1}{e^u + 2 + e^{-u}}$	
Sigmoid function	$K(u) = \frac{2}{\pi} \frac{1}{e^u + e^{-u}}$	

Mean-Shift Segmentation

- Finite-support kernel: [wiki/Truncated_normal_distribution]

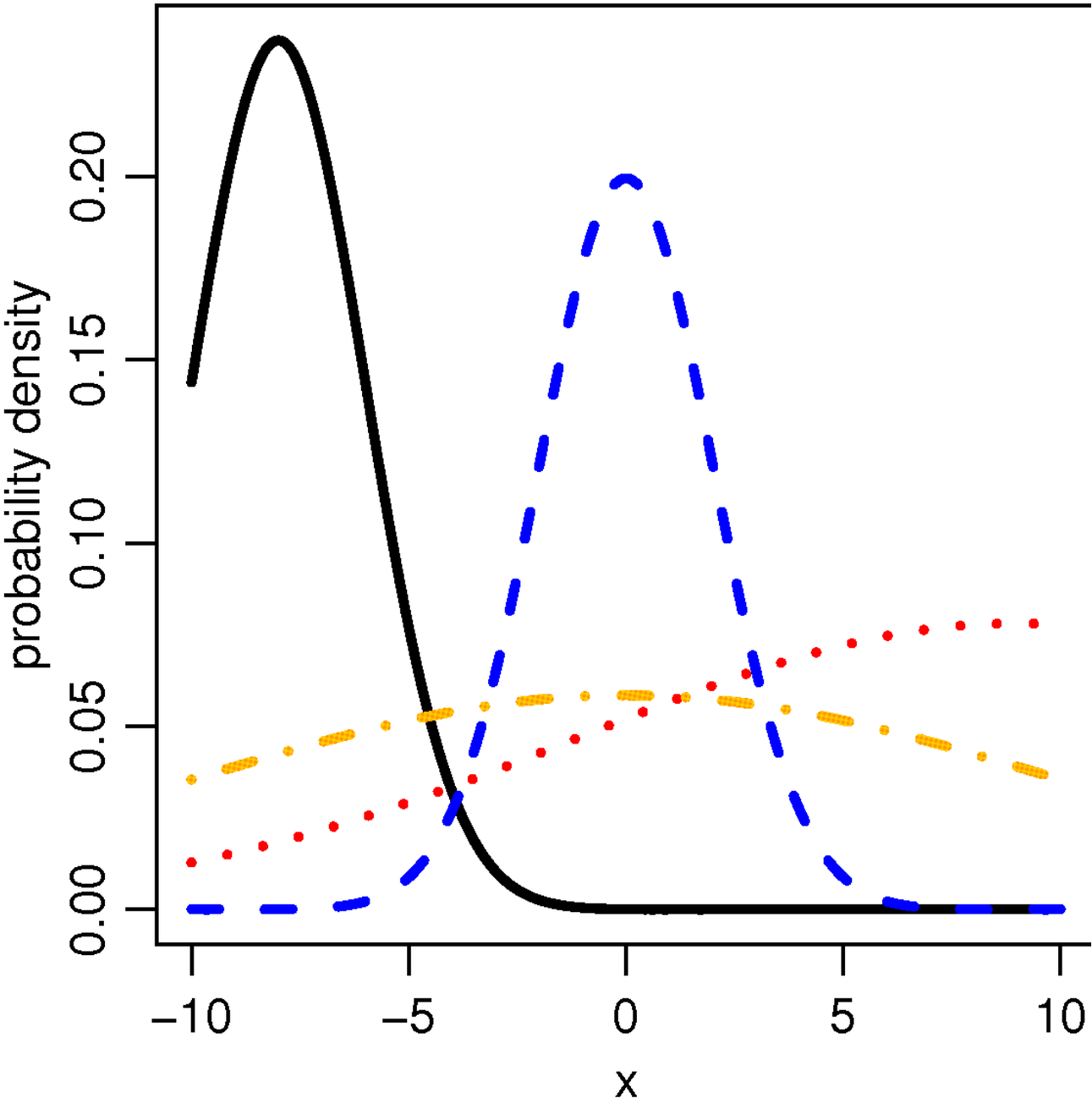
Its probability density function, f, for $a \leq x \leq b$, is

$$f(x;\mu,\sigma,a,b) = \frac{1}{\sigma}\frac{\varphi(\frac{x-\mu}{\sigma})}{\Phi(\frac{b-\mu}{\sigma}) - \Phi(\frac{a-\mu}{\sigma})}$$

and by $f = 0$ otherwise.

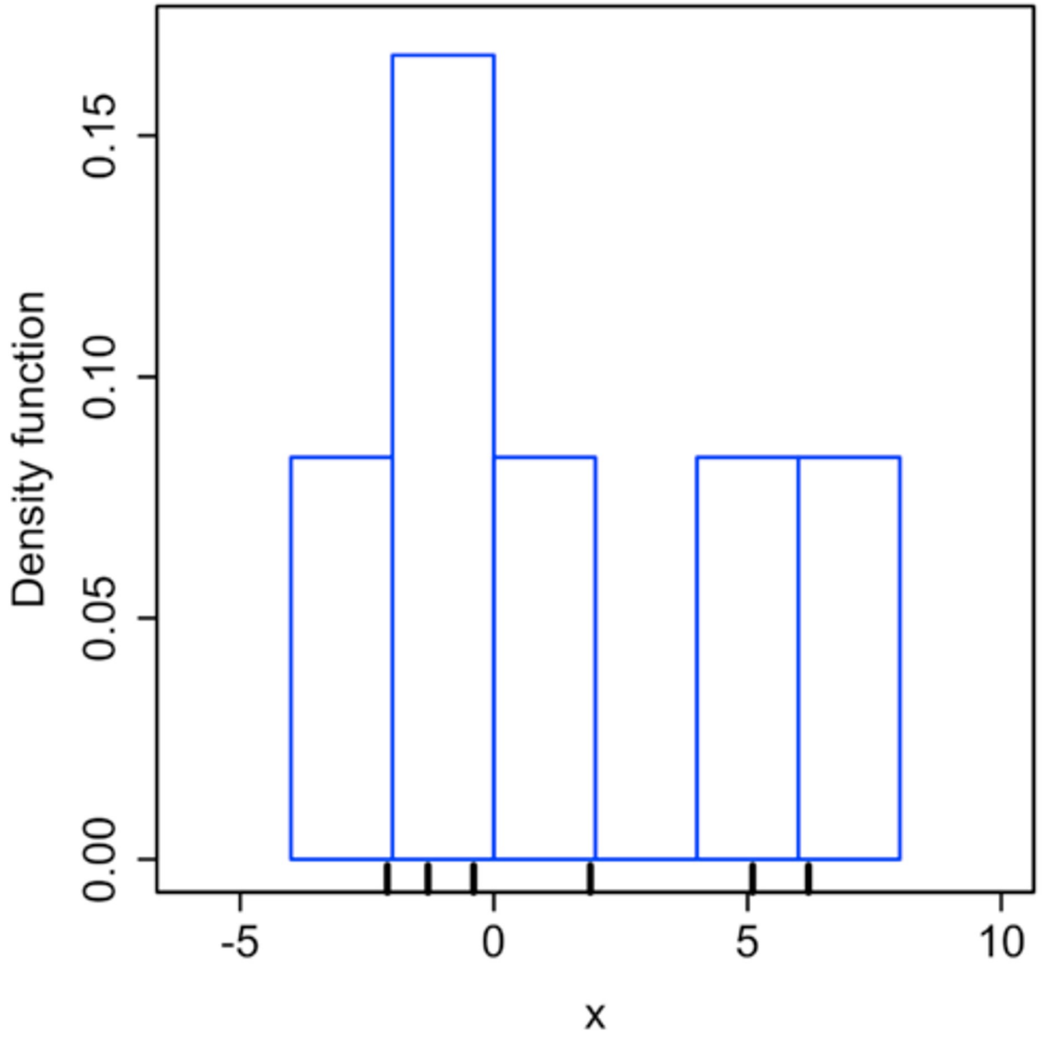
$$\varphi(\xi) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{1}{2}\xi^2\right)$$

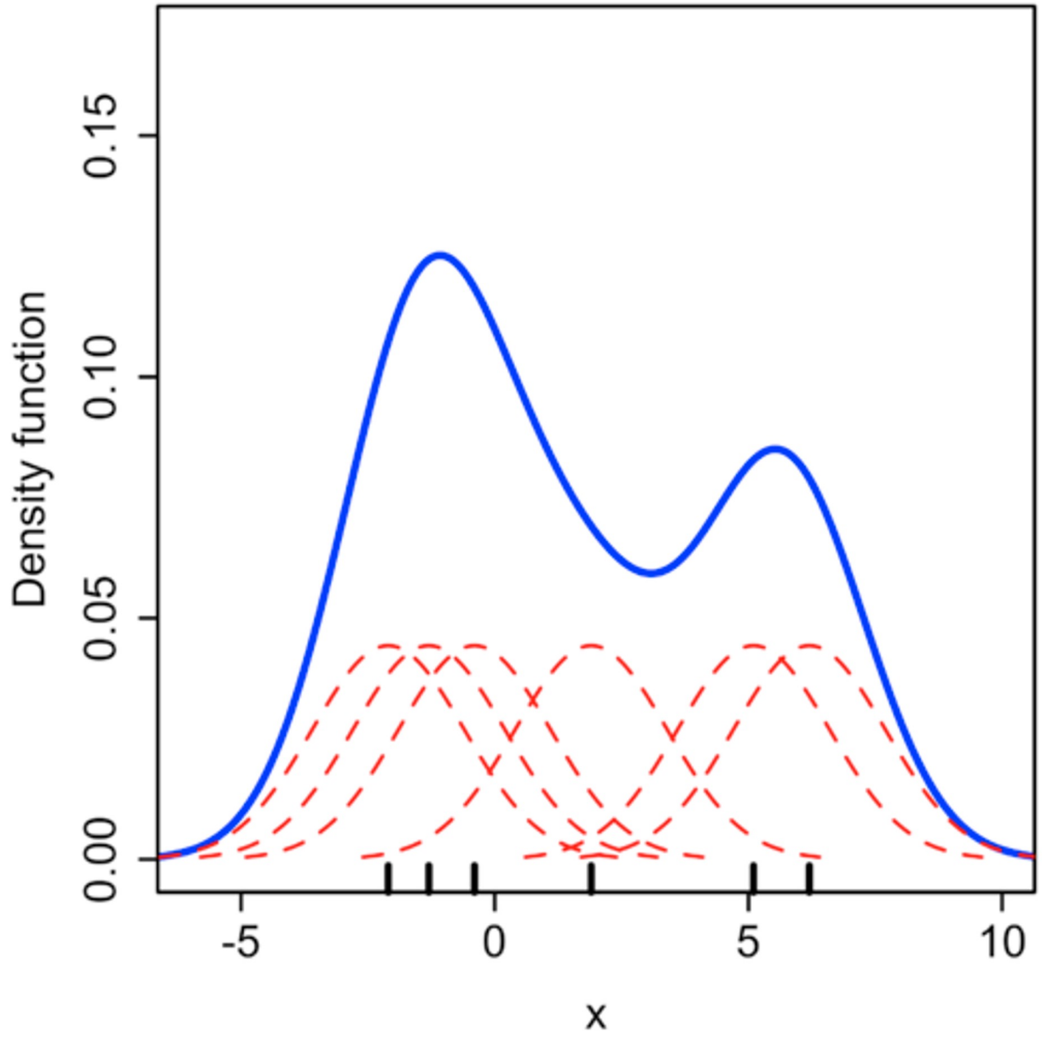
$$\Phi(x) = \frac{1}{2}\left(1 + \operatorname{erf}(x/\sqrt{2})\right)$$



Nonparametric/Kernel Density Estimation

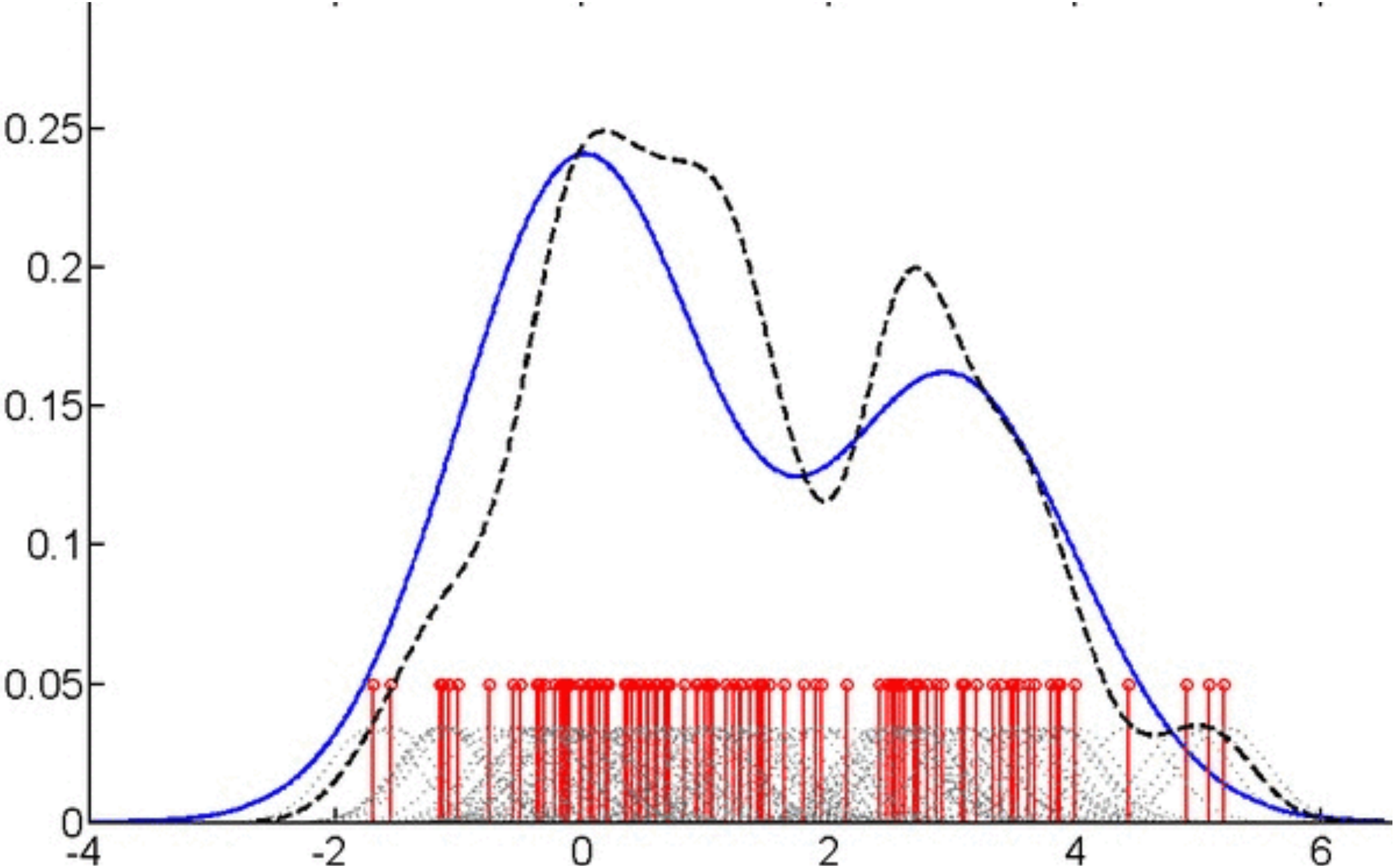
- Left: histogram
- Right: kernel density estimate (with Gaussian kernel)





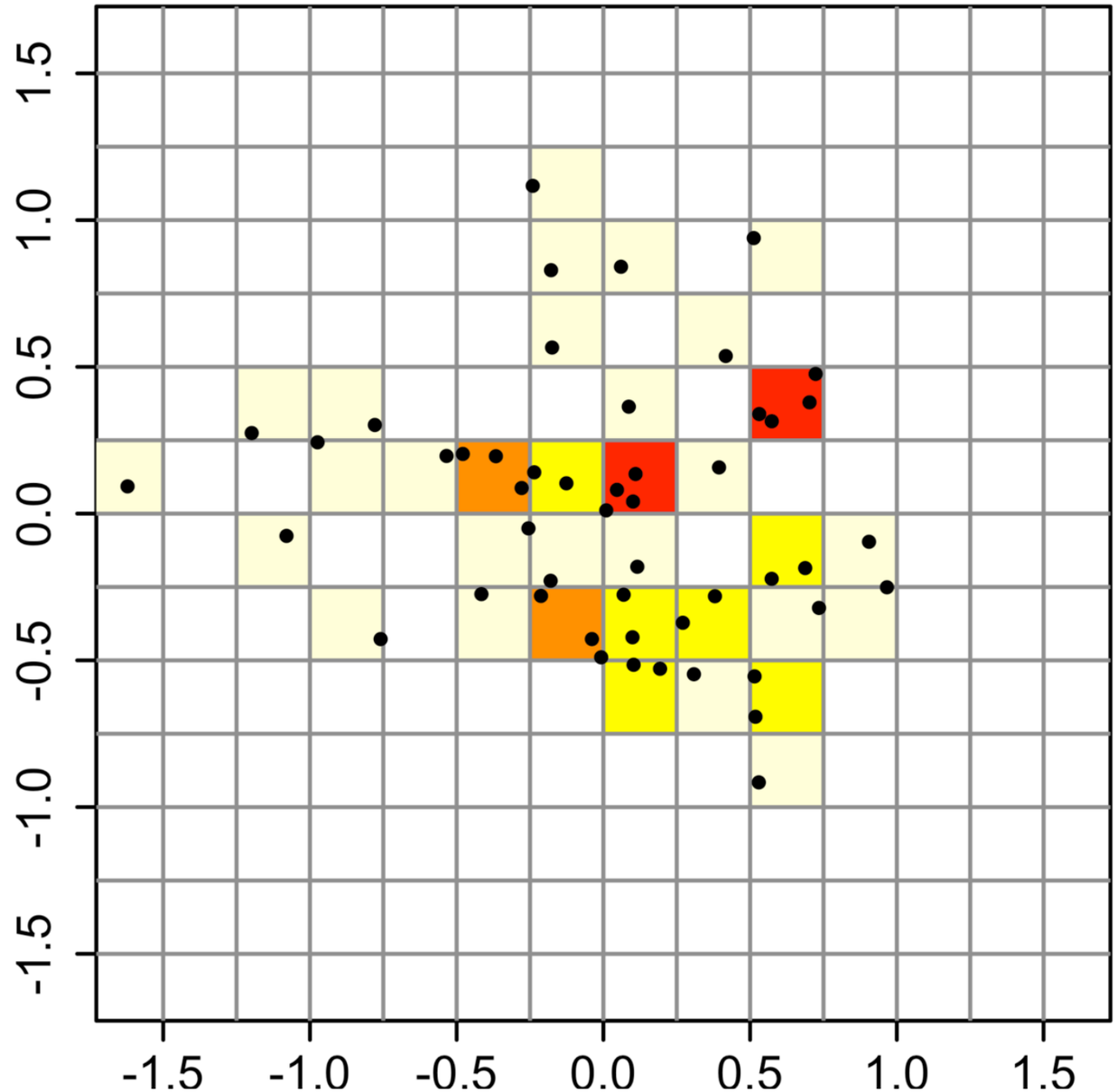
Nonparametric/Kernel Density Estimation

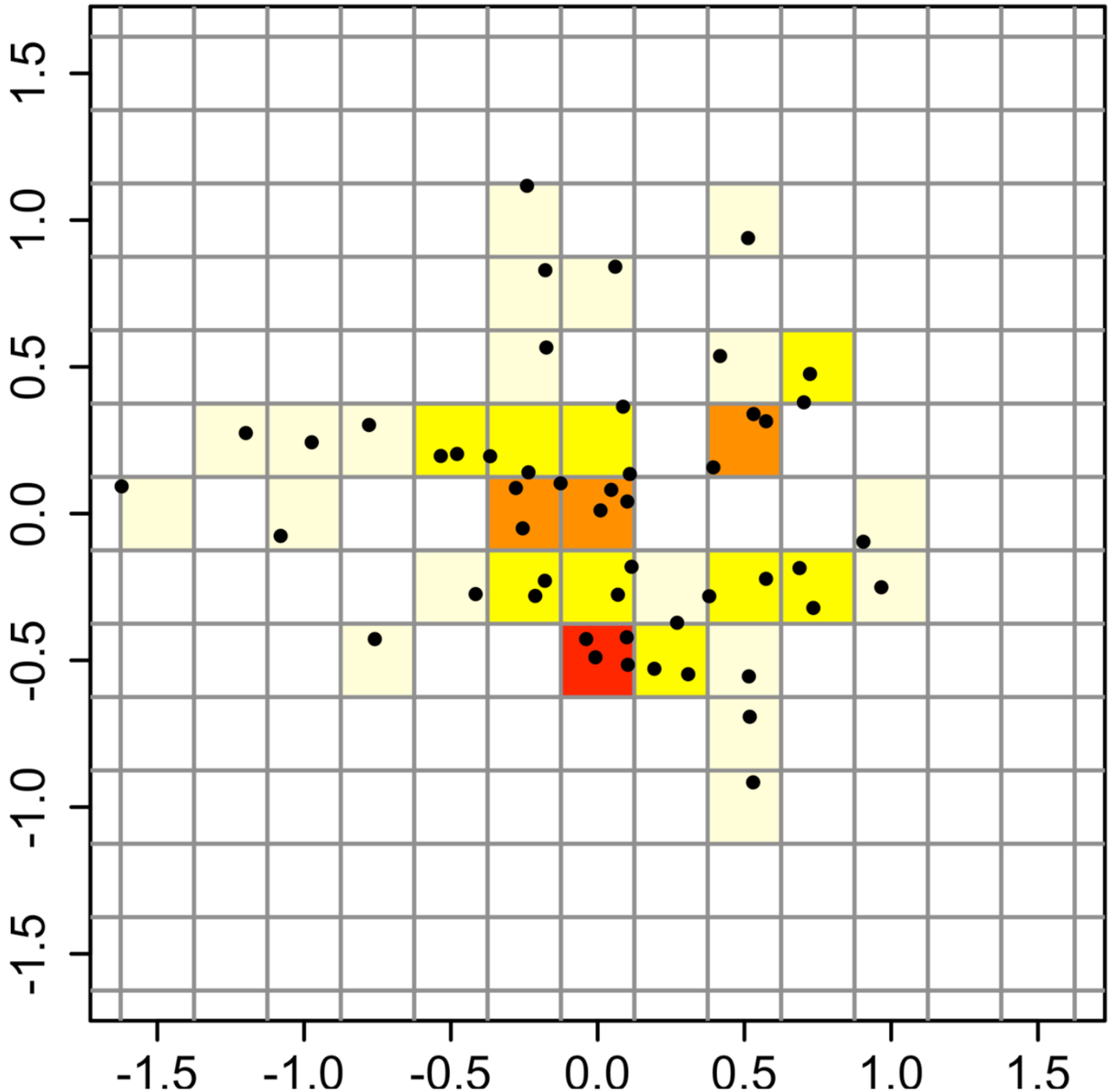
- Blue = true PDF
- Red = data set; i.i.d. drawn from PDF
- **Black** = kernel density estimate, for specific data set



Nonparametric/Kernel Density Estimation

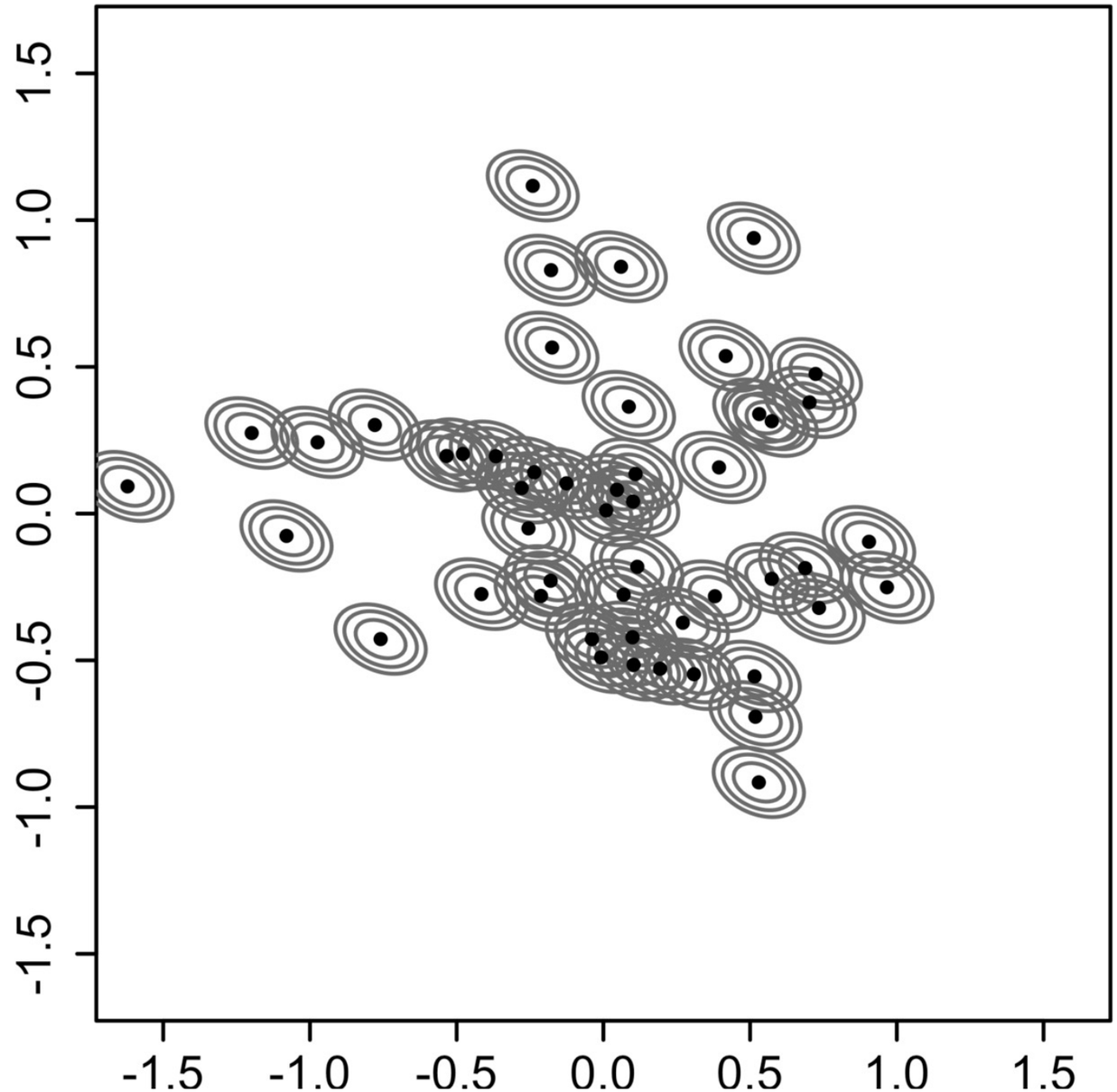
- Histogram (in 2D): depends on bin placement (and bin size)

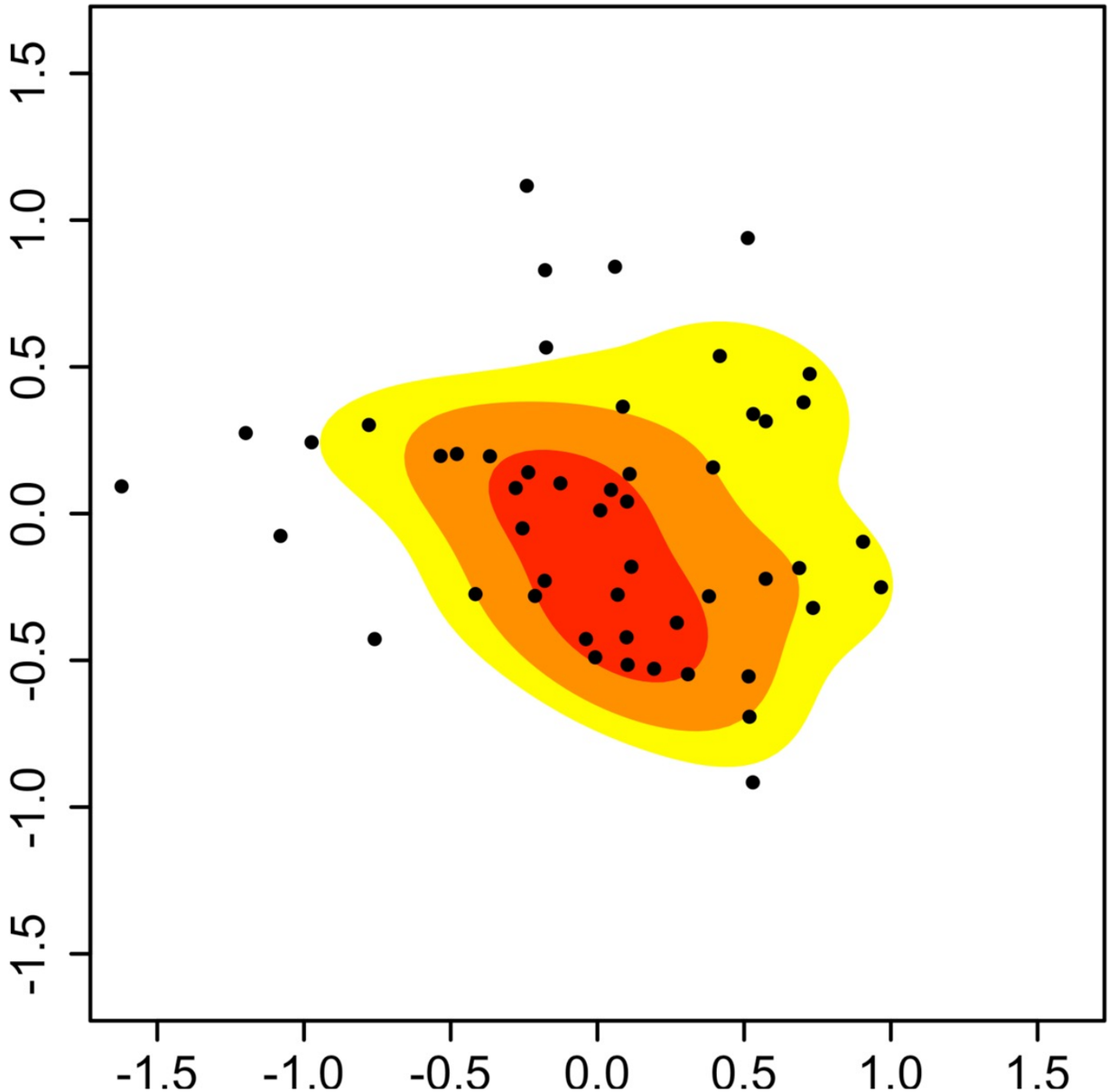




Nonparametric/Kernel Density Estimation

- Kernel density estimate: depends on kernel (co)variance





Nonparametric/Kernel Density Estimation

- Goal: Estimate probability density function (PDF) f(x) given observations { x_i in R^d : i = 1,…,n } drawn from f(x), where analytical/parametric form of f(x) is unknown
- Strategy: Superpose kernel functions, each placed at x_i
- h = scale/"bandwidth" parameter
- K(y) = kernel
  - e.g., standard normal $\exp(-0.5 \parallel y \parallel_2^2)/(2\pi)^{d/2}$
  - Non-negative, integrates to 1, mean 0, finite valued, decays to 0 sufficiently fast
  - Typically, radially symmetric
  - "Profile" (lowercase) k(.) = non-increasing; For Gaussian kernel: profile k(a) = exp (–a/2) for a>0

$$f(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^{n} K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$$

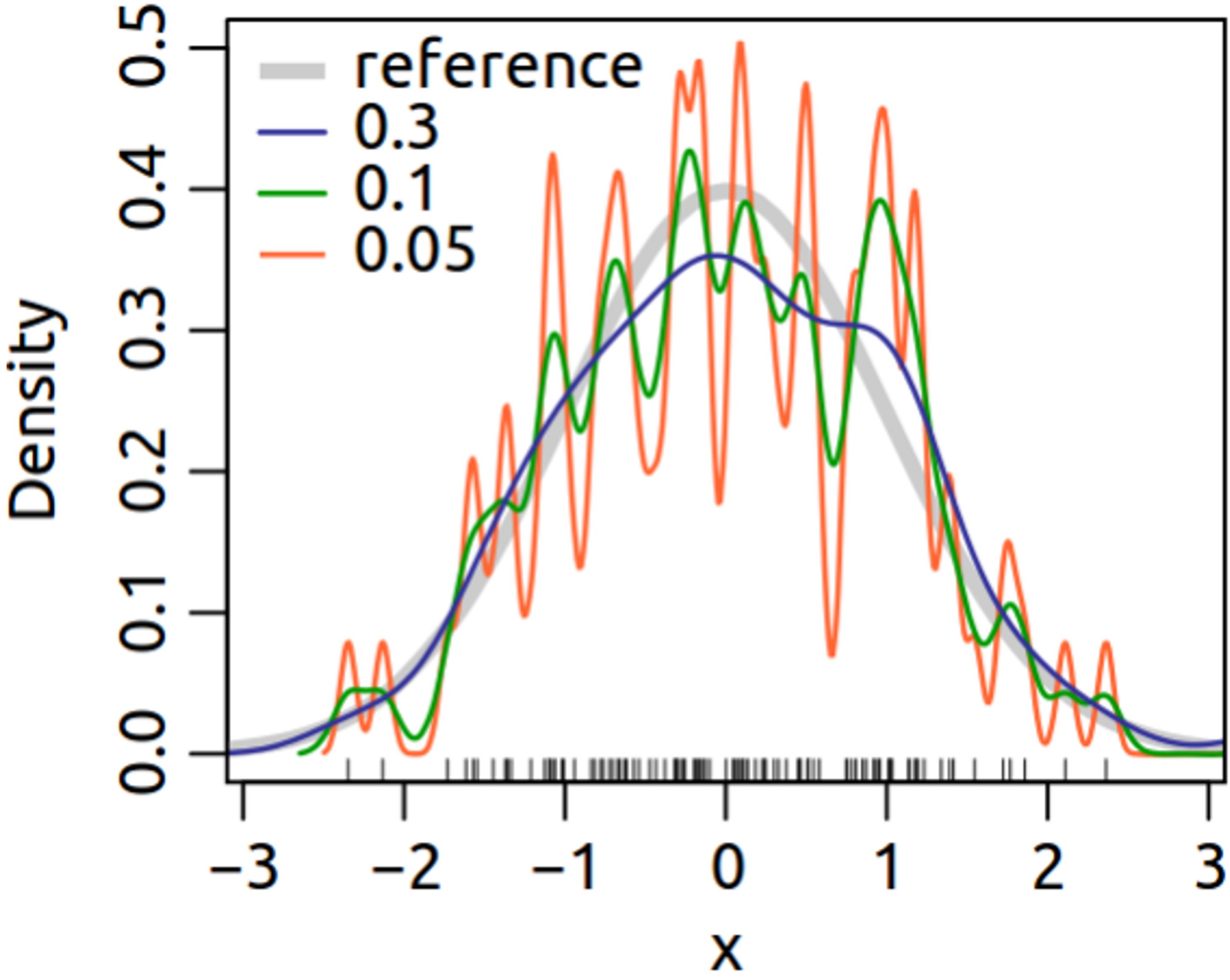
$$\int_{R^d} K(\mathbf{x}) \mathrm{d}\mathbf{x} = 1$$

$$\int_{R^d} \mathbf{x} K(\mathbf{x}) \mathrm{d}\mathbf{x} = 0$$

$$\lim_{\|\mathbf{x}\| \to \infty} \|\mathbf{x}\|^d K(\mathbf{x}) = 0$$

Nonparametric/Kernel Density Estimation

- Bandwidth/scale (h) selection
- **Too small: orange**
- **Small: green**
- **Desirable: blue**
- **True: gray**
- What if h is too large ?
- What if h → 0 ?



Nonparametric/Kernel Density Estimation

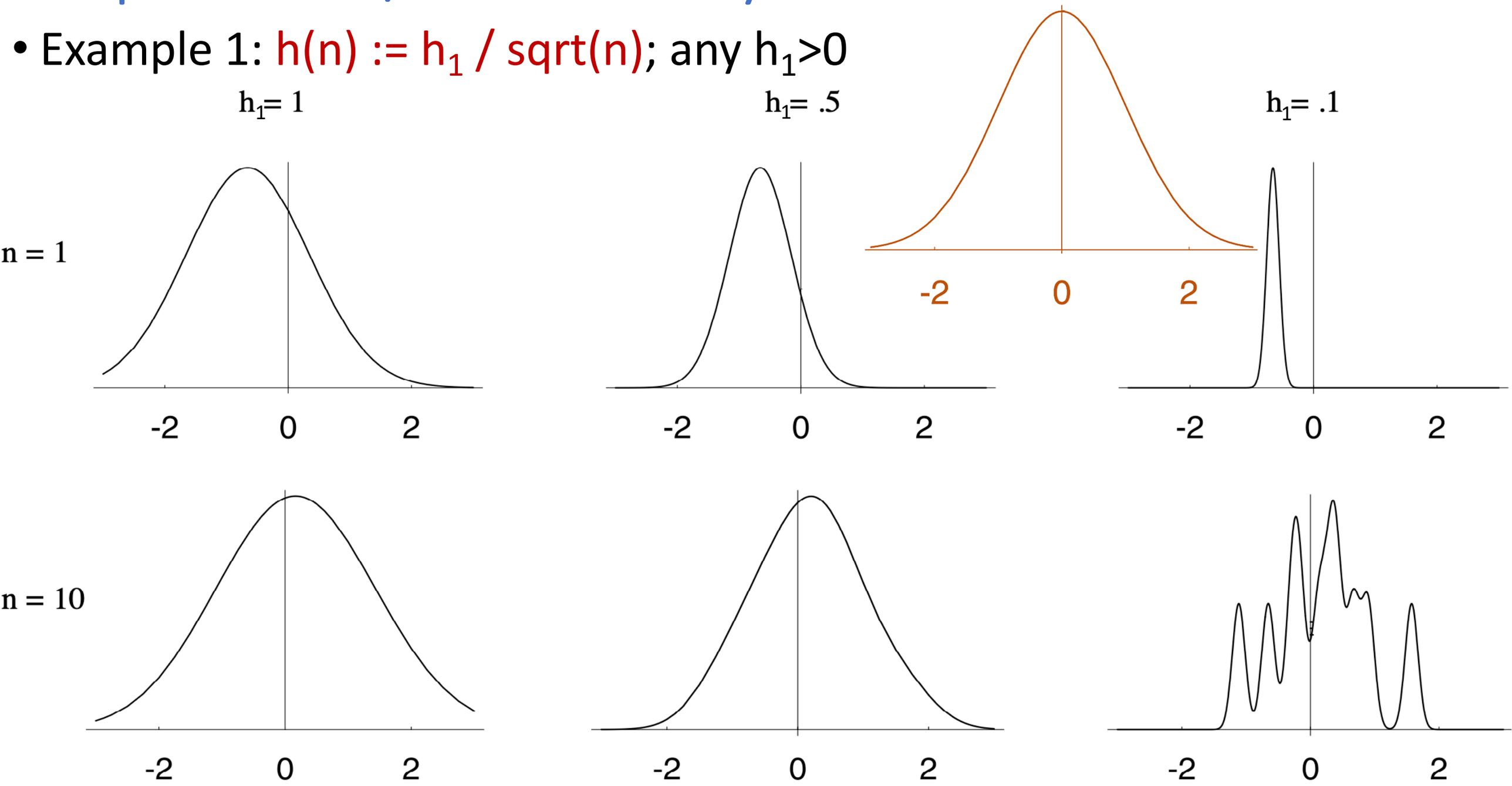
- How can PDF estimate converge to true PDF as sample size $n \to \infty$?
- Bandwidth constraints (motivation)

1) h must reduce with n (more data should need less smoothing)
2) $h(n) \to 0$ as $n \to \infty$ (else convolution always gives blurred estimate)
   - Sufficiently fast : $\mathbf{Lim_{n \to \infty} \; [\, h(n) \,]^d = 0}$
   - Sufficiently slow: $\mathbf{Lim_{n \to \infty} \; n \, [\, h(n) \,]^d = \infty}$ (decreases slower than 1/n)

- Convergence
  - Let random variable $P_n(x; X_1, \ldots, X_n)$ = kernel density estimate of PDF P(.) at x using a sample $\{ X_1, \ldots, X_n \}$ of cardinality n
  - Then, $\lim_{n \to \infty} E_{P(X1, \ldots, Xn)} [\, P_n(x; X_1, \ldots, X_n) \,] = P(x)$
    - Expectation is w.r.t. sampling distribution, of n-sized sample drawn from P(.)
  - And, $\lim_{n \to \infty} \text{Var} [\, P_n(x) \,] = 0$ (mean-square convergence)

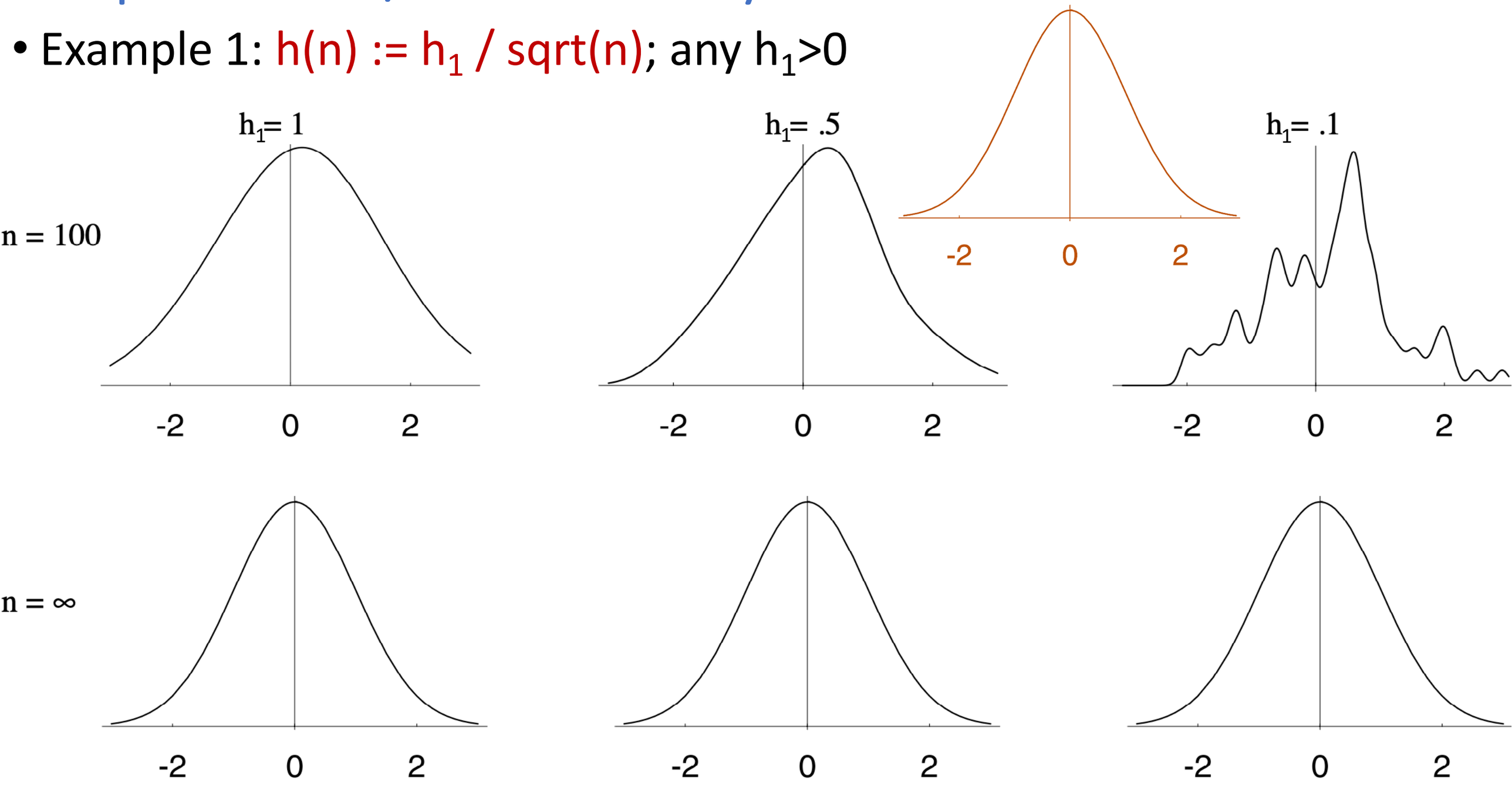
Nonparametric/Kernel Density Estimation

- Example 1: h(n) := h_1 / sqrt(n); any h_1>0



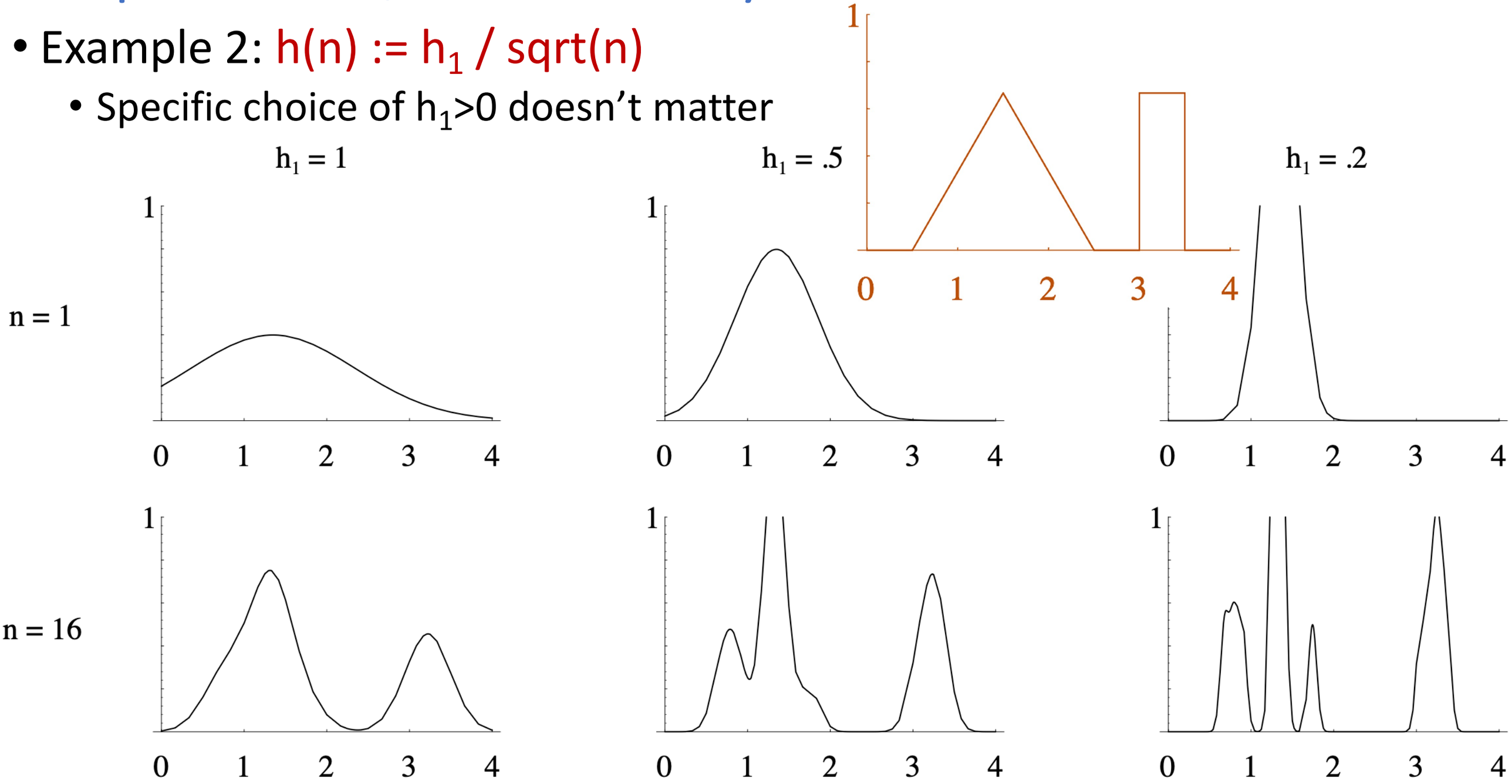
Nonparametric/Kernel Density Estimation

- Example 1: h(n) := h_1 / sqrt(n); any h_1>0



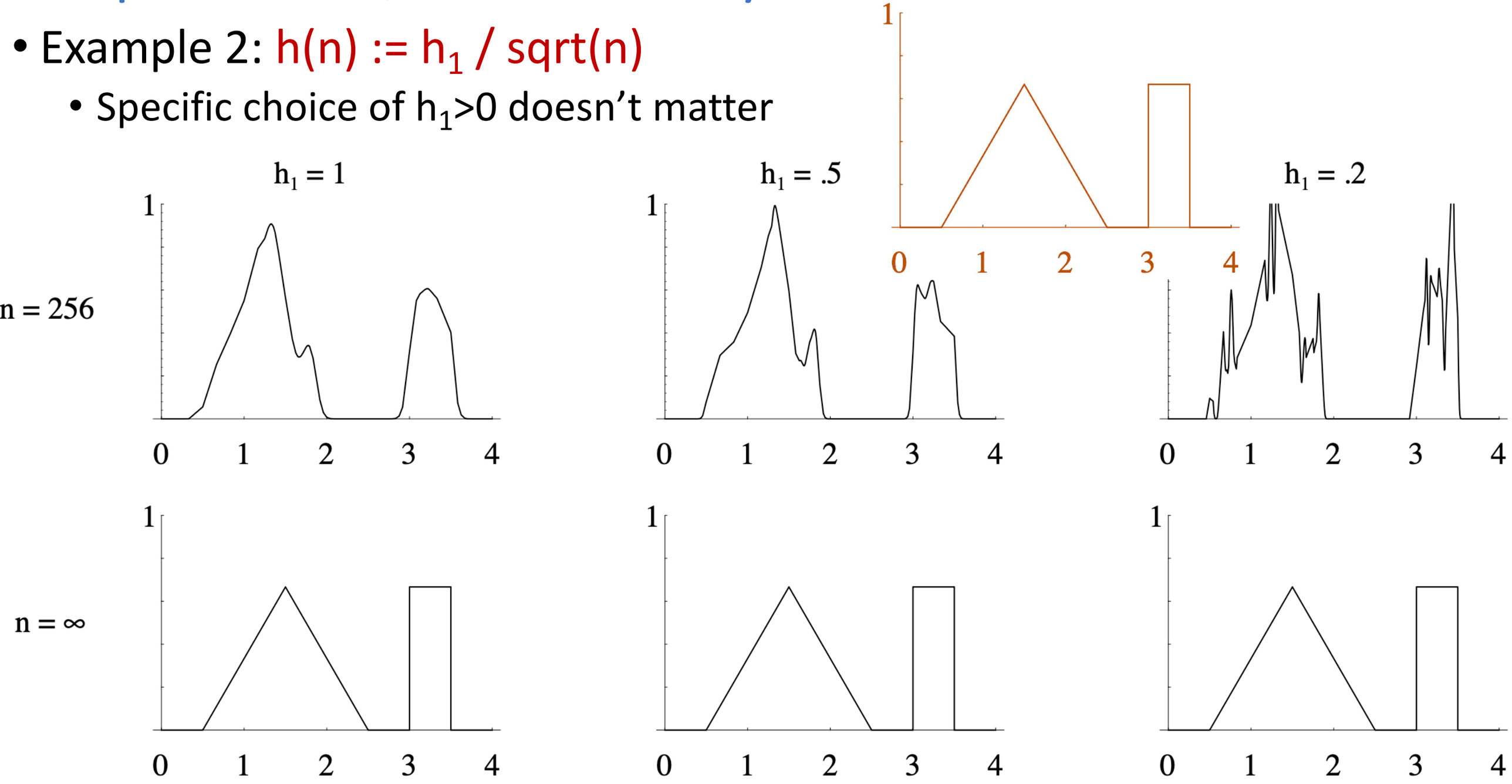
Nonparametric/Kernel Density Estimation

- Example 2: h(n) := h_1 / sqrt(n)
  - Specific choice of h_1>0 doesn't matter



Nonparametric/Kernel Density Estimation

- Example 2: h(n) := h_1 / sqrt(n)
  - Specific choice of h_1>0 doesn't matter



Nonparametric/Kernel Density Estimation

- Convergence of expectation/mean of PDF estimate
- Random variable $p_n(x)$:= PDF estimate at x with sample size n
  := $(1/n) \sum_i (1/V_n)\ \varphi((x–x_i)/h_n)$ (e.g., $\varphi(.)$ is standard normal)
  where $V_n := (h_n)^d$
- Inside each expectation,
  x_i sampled from same p(X)
- So, all n expectations are same = $\bar{p}_n(x)$
- So, LHS = E [$(1/V_n)\ \varphi((x–x_i)/h_n)$]
- = Conv (PDF p(.), kernel $\varphi(./h_n)/V_n$)

$$\bar{p}_n(\mathbf{x}) = \mathcal{E}[p_n(\mathbf{x})]$$

$$= \frac{1}{n}\sum_{i=1}^{n} \mathcal{E}\left[\frac{1}{V_n}\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right)\right]$$

$$= \int \frac{1}{V_n}\varphi\left(\frac{\mathbf{x}-\mathbf{v}}{h_n}\right) p(\mathbf{v})\ d\mathbf{v}$$

- If $V_n \to 0$, kernel → impulse function & convolution converges to p(x); this happens even for finite n (good; but what about variance ?)
  - If $V_n > 0$, then KD estimate is biased; for finite n, we prefer biased but smooth

Nonparametric/Kernel Density Estimation

- Variance of PDF estimate
- Random variable $p_n(x) := \sum_i \varphi((x–x_i)/h_n) / (nV_n)$ = sum of functions of independent x_i
- So, Var ($p_n(x)$) = sum of variances of its terms
- Each term's variance is same

$$\sigma_n^2(\mathbf{x}) = \sum_{i=1}^{n} \mathcal{E}\left[\left(\frac{1}{nV_n}\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right) - \frac{1}{n}\bar{p}_n(\mathbf{x})\right)^2\right]$$

$$= n\,\mathcal{E}\left[\frac{1}{n^2V_n^2}\varphi^2\left(\frac{\mathbf{x}-\mathbf{x}_i}{h_n}\right)\right] - \frac{1}{n}\bar{p}_n^2(\mathbf{x})$$

$$= \frac{1}{nV_n}\int \frac{1}{V_n}\varphi^2\left(\frac{\mathbf{x}-\mathbf{v}}{h_n}\right) p(\mathbf{v})\, d\mathbf{v} - \frac{1}{n}\bar{p}_n^2(\mathbf{x})$$

- Now $\varphi^2(.) \le \varphi(.)\ \sup(\varphi(.))$
- So, integral $\le \sup(\varphi(.))\ E[p_n(x)]$
- So, $0 \le$ LHS $\le \sup(\varphi(.))\ E[p_n(x)] / (nV_n) – E[p_n(x)]^2 / n$

Nonparametric/Kernel Density Estimation

- Variance of PDF estimate
- We have: $0 \le \text{Var}(p_n(x)) = \text{LHS} \le \sup(\varphi(.))\, E[p_n(x)] / (nV_n) - E[p_n(x)]^2 / n$

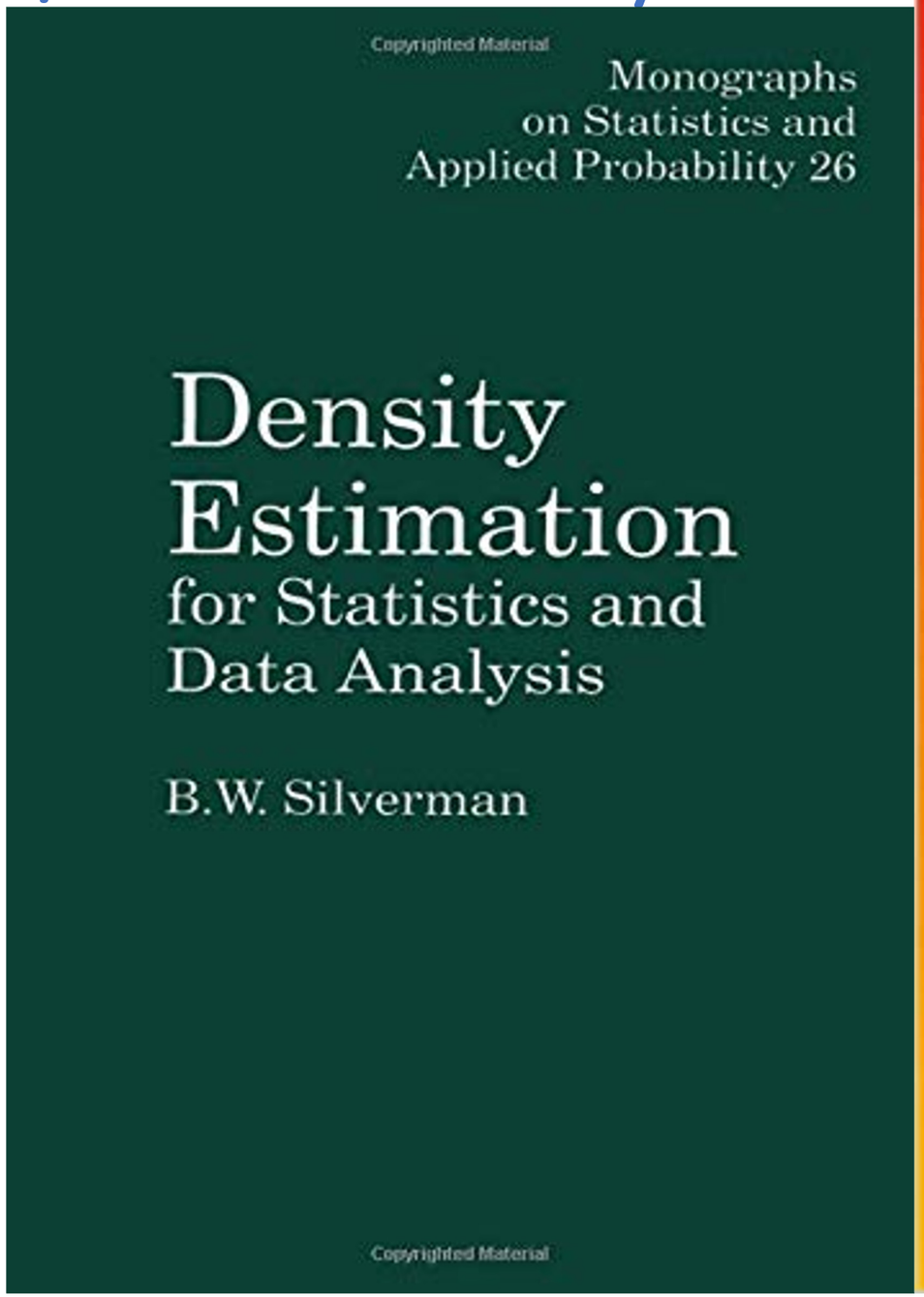
- As $n \to \infty$, earlier, we let $V_n \to 0$ to make mean of $p_n(x)$ converge to $p(x)$

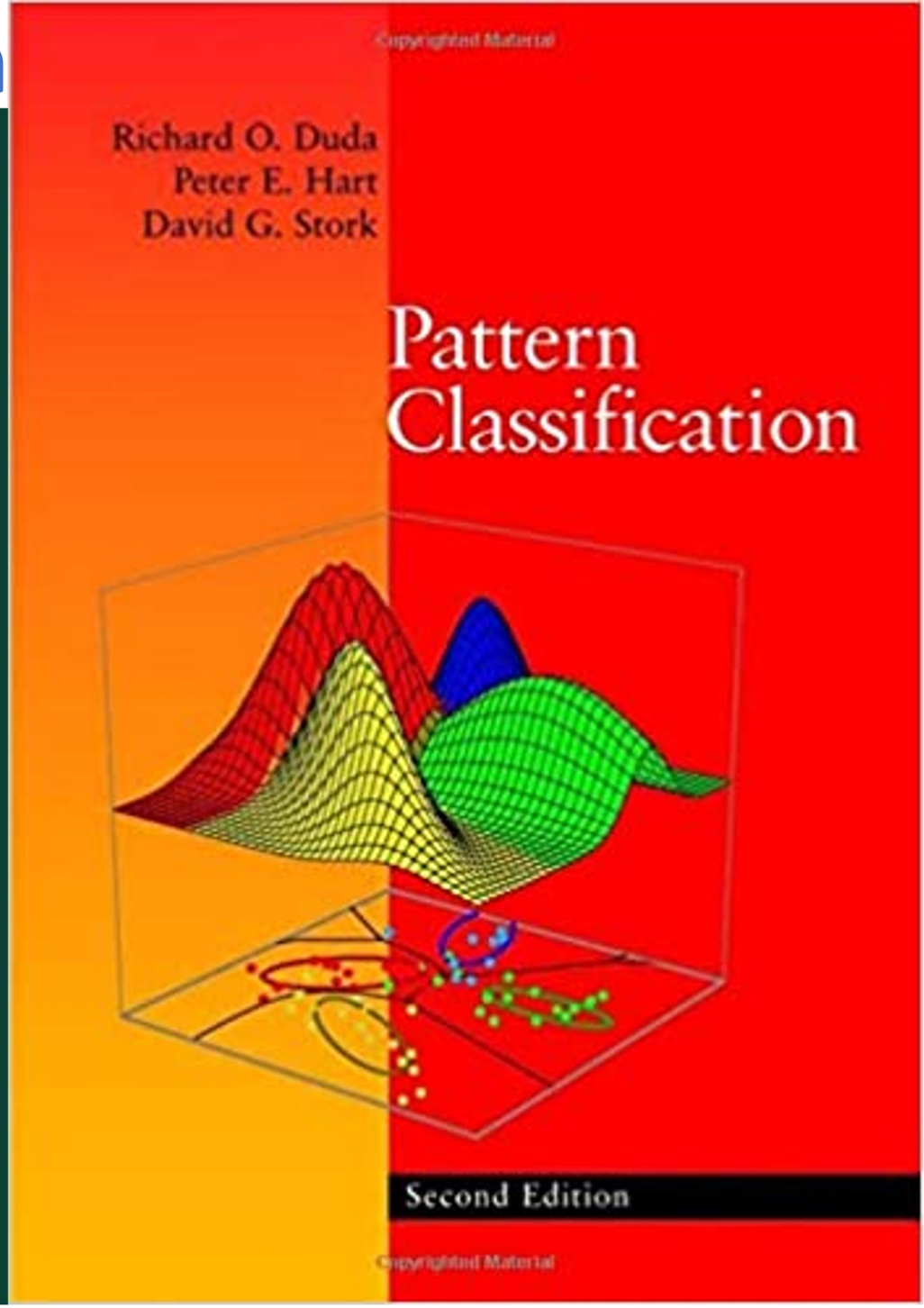
- So, as $n \to \infty$ & $V_n \to 0$, second term $\to 0$ (assume $E[p_n(x)] = p(x)$ finite)

- Also, as $n \to \infty$, we want first term $\to 0$
  to achieve zero variance of $p_n(x)$
  for which we can let $nV_n \to \infty$
  - $\sup(\varphi(.))$ is finite, e.g., when $\varphi(.)$ is standard normal
- Typical choices for V_n are $V_n := V_1 / \log(n)$ or $V_n := V_1 / \text{sqrt}(n)$

Nonparametric/Kernel Density Estim

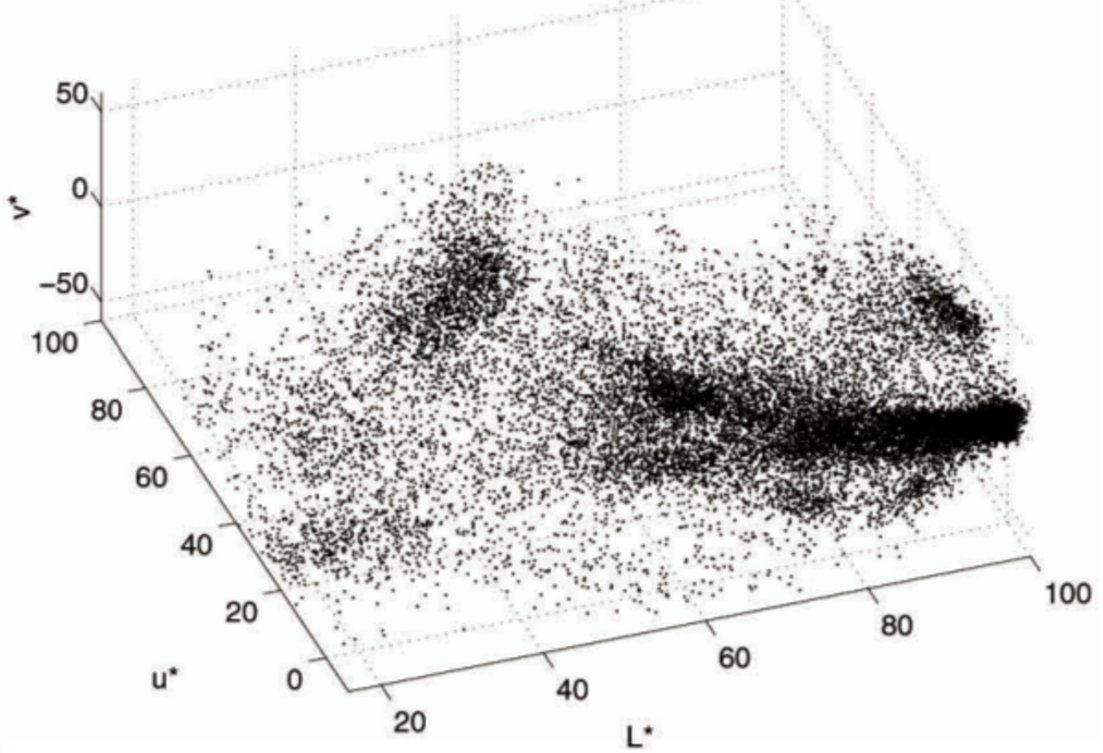
- Books
- Notes: link





Mean-Shift Segmentation

- Left: color image, 3 color components (R,G,B) per pixel
- Right: Scatter plot of color 3-tuples (L*,u*,v*) derived from (R,G,B)
- Assumption: Each “object” → “cluster” of color values
- At each pixel, L*u*v* can be replaced by other higher-dim features, e.g., NxN patch, M filter outputs to characterize local texture, etc.



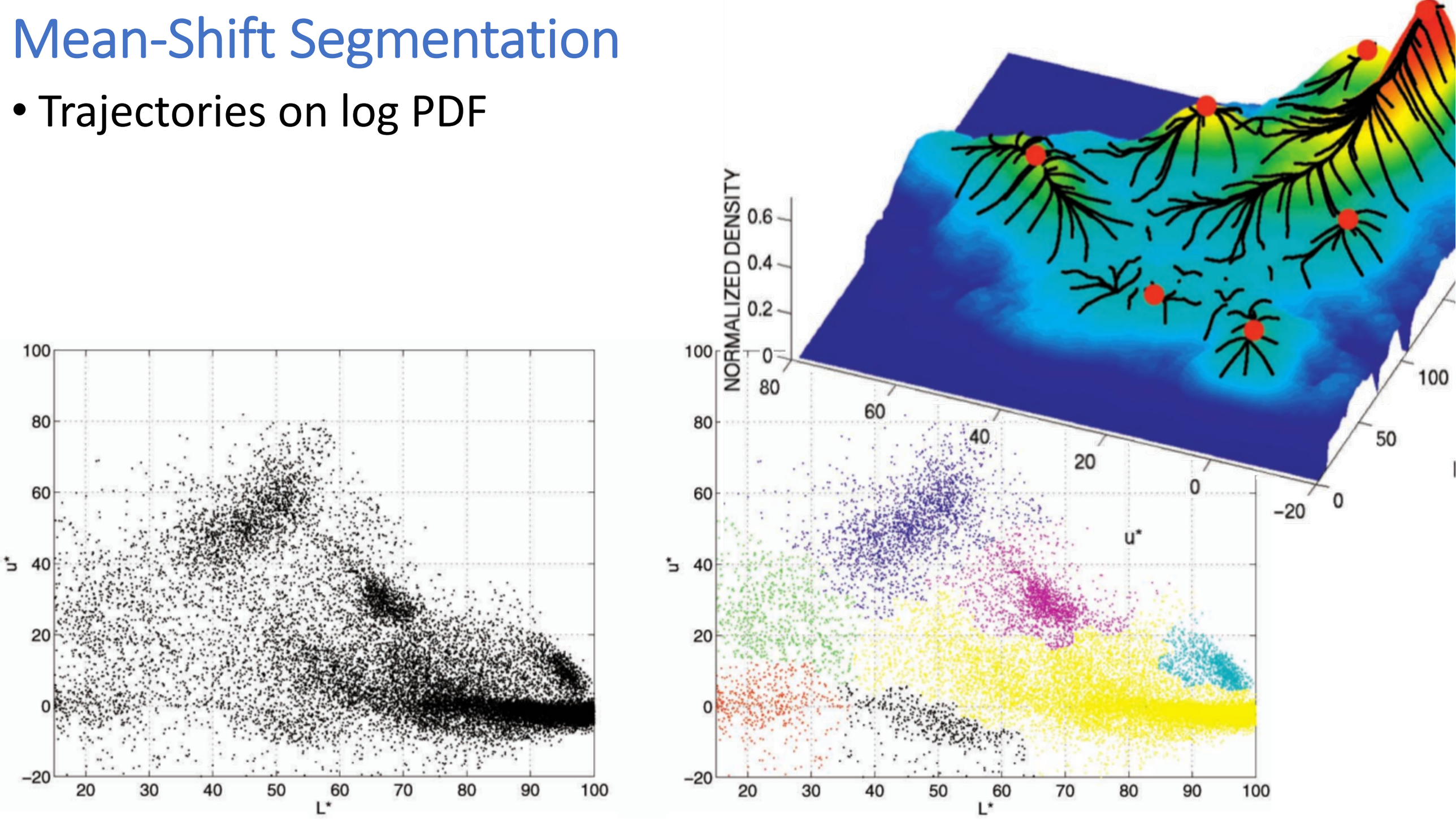
Mean-Shift Segmentation

- Algorithm:

1) Choose bandwidth h: user-defined/“free” parameter
2) Use h and (multi-dimensional) colors/feature values
   { x_i at pixel index i } to estimate PDF f(.)
3) For each x_i, do following:
   - Compute gradient of log(f(.)) at x_i
   - Update/shift x_i via step along gradient direction (notion of “mean”, later)
   - Each x_i moves towards local mode of (estimated) PDF f(.)
4) Option 1: PDF estimate f(.) isn’t updated, OR
   Option 2: PDF estimate f(.) is updated using updated { x_i }
   - Option 2 makes updated PDF f(.) more concentrated around local modes
5) Repeat last 2 steps, until termination criterion (described later)

Mean-Shift Segmentation

- Trajectories on log PDF



Mean-Shift Segmentation

- Evaluate gradient of log-PDF
- PDF (with Gaussian kernel):

$$f(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^{n} K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right)$$

  - Recall: profile k(a) := exp(–a/2) for a>0
  - For Gaussian kernel: denote k(.) by g(.)

$$K(\mathbf{x}) = c_{k,d} k(\|\mathbf{x}\|^2)$$

- Gradient of f(.) at x:

$$\nabla f(\mathbf{x}) = \frac{2c_{k,d}}{nh^{d+2}} \sum_{i=1}^{n} (\mathbf{x}_i - \mathbf{x}) g\left(\left\|\frac{\mathbf{x} - \mathbf{x}_i}{h}\right\|^2\right)$$

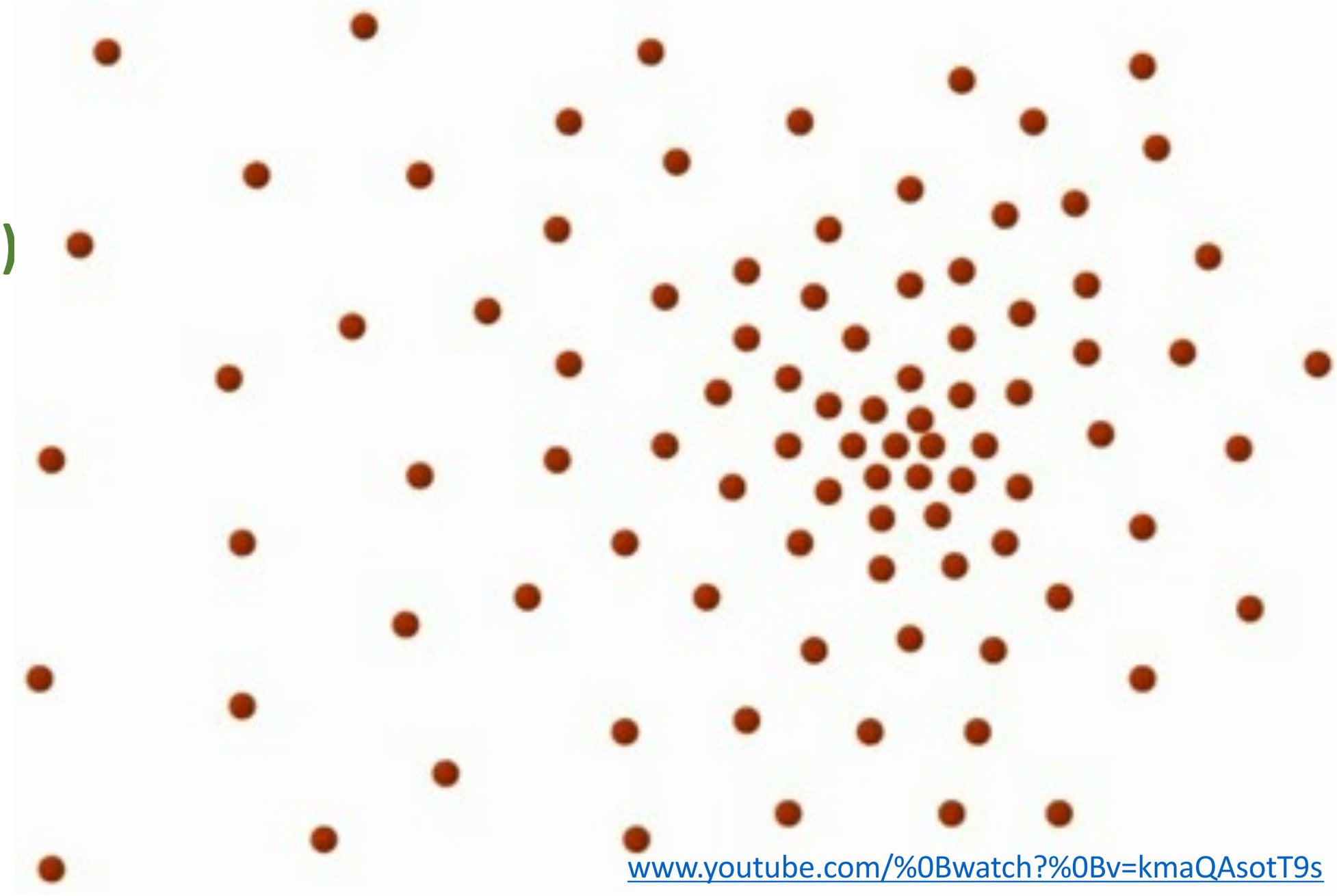
- Gradient of log(f(.)) at x is ∝

$$\left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)}{\sum_{i=1}^{n} g\left(\left\|\frac{\mathbf{x}-\mathbf{x}_i}{h}\right\|^2\right)} - \mathbf{x}\right]$$

- Choose step size ∝ $h^2/2$
- This shifts x to kernel-weighted mean of { all x_i }; weights depend on x; weights positive, sum to 1

Mean-Shift Segmentation

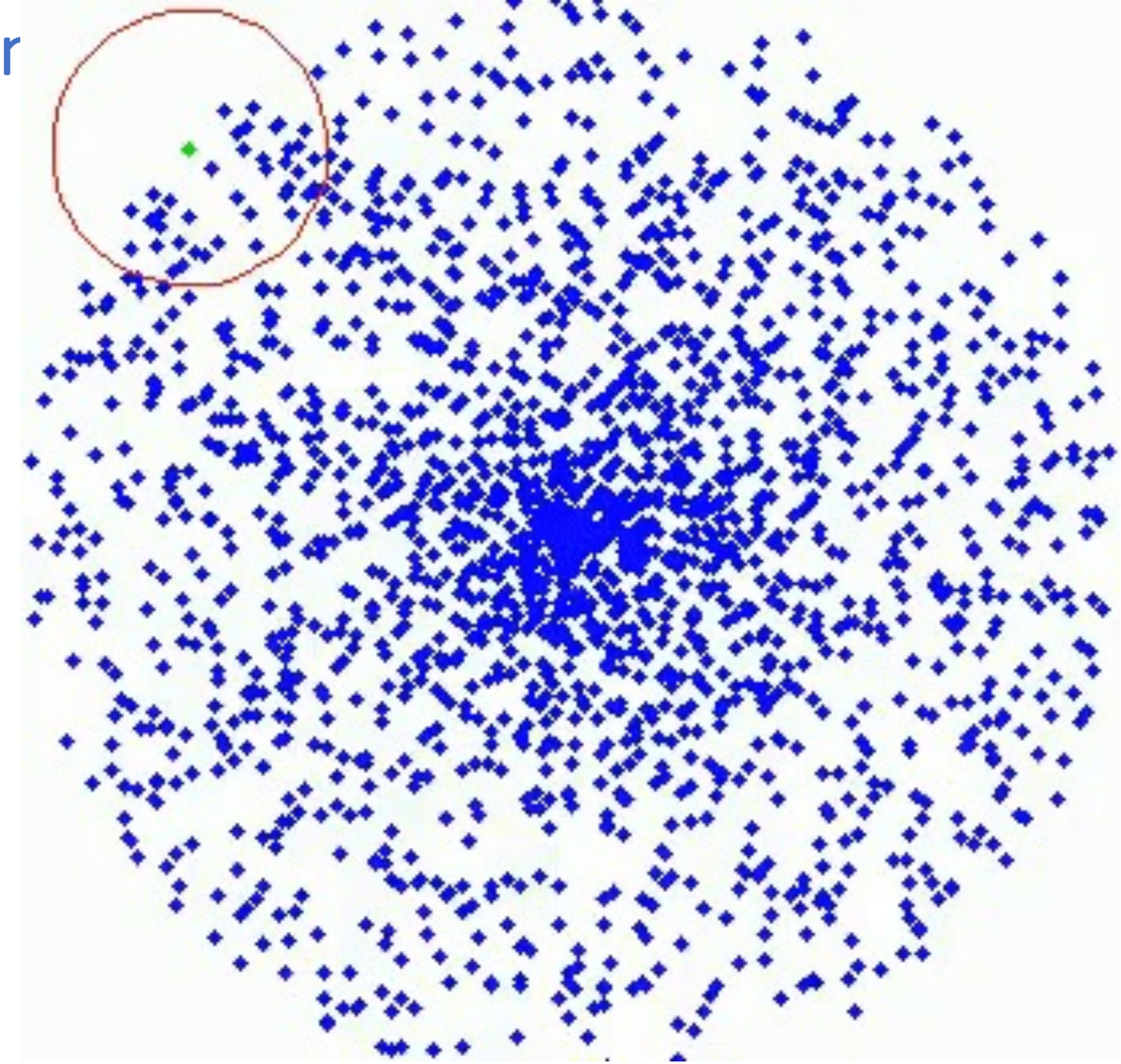
- Mean-shift updates
- As an example, 1 point **x (green)** moves
- PDF estimate f(.), obtained from red points, stays fixed



www.youtube.com/%0Bwatch?%0Bv=kmaQAsotT9s

Mean-Shift Segmentation

- Mean-shift updates
- As an example,
  1 point **x (green)** moves
- PDF estimate f(.),
  obtained from blue points,
  stays fixed



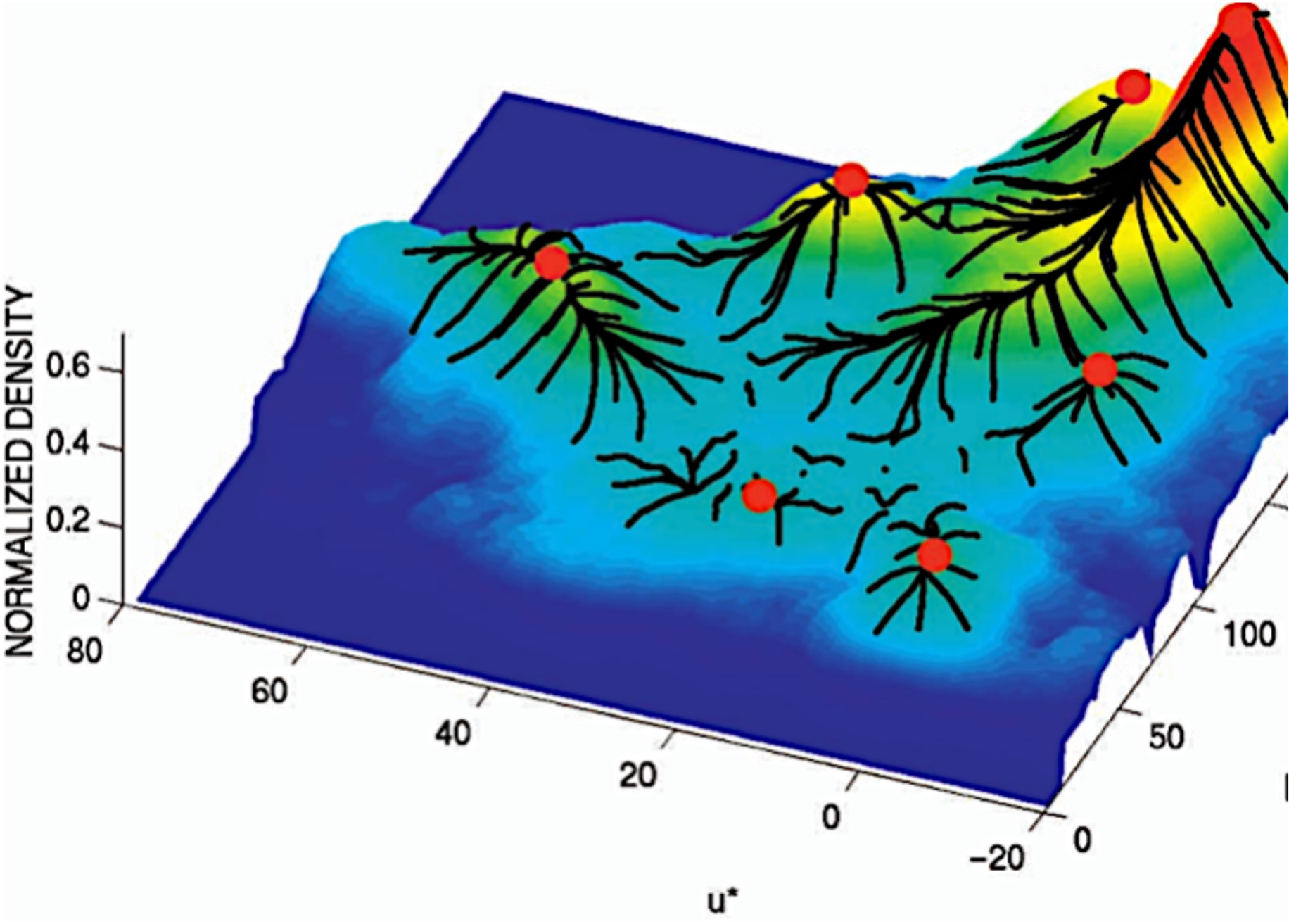
www.youtube.com/%0Bwatch?v=hJg7ik4x95U

Mean-Shift Segmentation

- Cluster results to get segmentation
  - Assign same label to group of points coalesced (very close) in feature space
  - Clustering (based on some tolerance for variability within each segment, e.g., due to finite-precision arithmetic)
- Why does mean-shift use gradient of log-PDF (instead of PDF) ?
  1) When using Gaussian kernel, updates interpretable as mean shift
  2) Updates become “slower” as we climb up towards/near local modes of f(.)
     - Gradient gets scaled down by larger f(.)
     - Can help avoid overshooting peak (local mode) when peaks are tall and sharp, which will happen if PDF estimate f(.) keeps updating

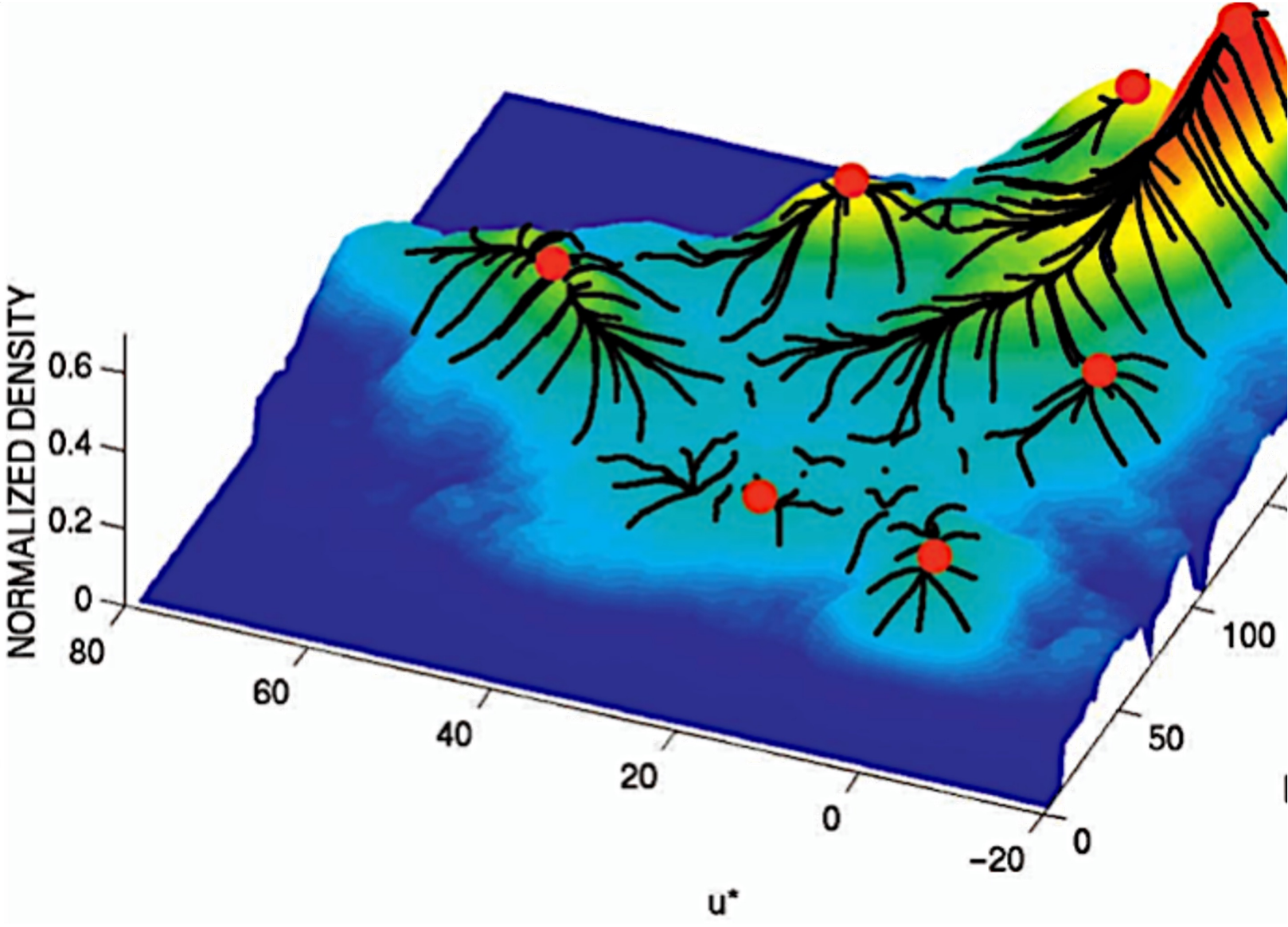
Mean-Shift Segmentation

- Trajectories on log PDF
- What is steady state (after ∞ iterations) with kernel of **infinite** support, e.g., Gaussian ?



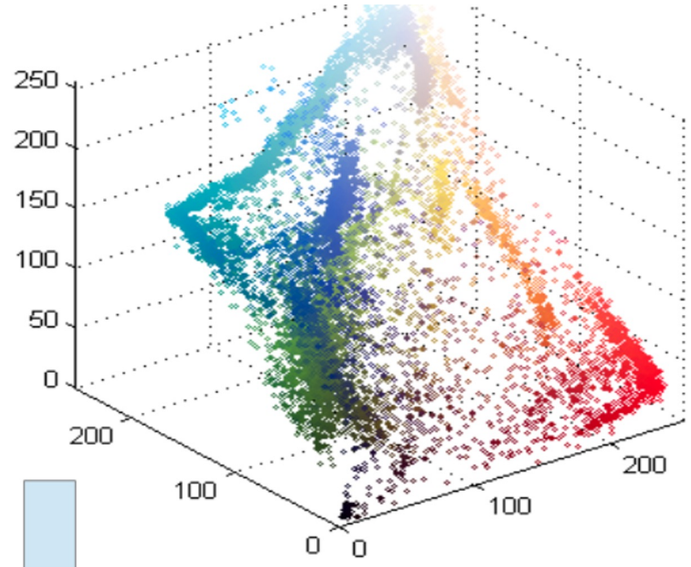
Mean-Shift Segmentation

- Trajectories on log PDF
- How to achieve this (in picture) as steady state ?
- Kernel needs to have **finite** support
  - e.g., truncated normal
  - If Epanechnikov, then gradient ascent on PDF (not log-PDF) (step $\propto h^{d+2}$) leads to update towards local average in feature space



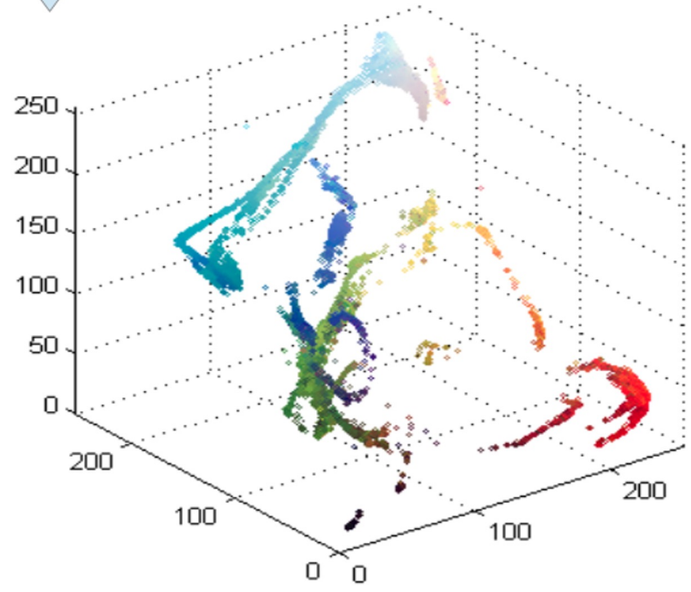
Mean-Shift Segmentation

- Algorithm in action



output image

Pixel Distribution After Meanshift



Mean-Shift Segmentation

- How to differentiate objects of similar color that are spatially apart ?
- Need kernel on joint space of pixel's color/feature + pixel's spatial location

$$K_{h_s,h_r}(\mathbf{x}) = \frac{C}{h_s^2 h_r^p} k\left(\left\|\frac{\mathbf{x}^s}{h_s}\right\|^2\right) k\left(\left\|\frac{\mathbf{x}^r}{h_r}\right\|^2\right)$$

  - Product of kernels: one in color/feature domain and another in spatial domain
  - 2 bandwidth parameters: h_s ("s" for spatial) & h_r ("r" for feature/range)

  - Alternately, we could also:
    1) rescale one of the features (colors-based or spatial-coordinate-based) and
    2) choose same kernel bandwidth parameter h for both features
    - We still have 2 free parameters (1 scaling, 1 bandwidth h)

Mean-Shi

Mean-Sh

Mean-Shift Segmentation

Mean-Shift Segmentation

- Effect of bandwidth
- Increase h_r → more color mixing
- Increase h_s → more spatial mixing

$(h_s, h_r) = (8, 8)$

$(h_s, h_r) = (8, 16)$

$(h_s, h_r) = (16, 8)$

Mean-Shift Segmentation

- Why not K-means ?

1) K-means models cluster shapes to be spherical
2) How to select K ?

- Mean shift implicitly does so by selecting bandwidth ‘h’
- Selecting ‘h’ may be more intuitive than selecting K

3) K-means doesn’t use PDF-gradient information

- Depending on initialization,
  K-means can put points around a local mode into 2 clusters;
  Given sufficiently large h, mean-shift won’t do this.
  Given sufficiently small K, k-means won’t do this